

Procedural content generation game level zelda legend

Table of Contents

Procedural content generation in video games (background)

- Procedural content generation (PCG) in games: definitions and scope

- Zeldalike endtoend world generation: scientific motivation and relevance

- Evolutionary algorithms for PCG game levels: typical representations and operators

- Identifying research gaps and defining research questions for endtoend systems

 - Literature review strategy for procedural content generation and level design

 - Translating gaps into precise research questions for endtoend systems

- PCG game level design: literature to prototype workflow (papers, tutorials, next steps)

Zelda-like PCG: Key constraints and evaluation metrics

- Key/lock progression modeling for guaranteed solvability

 - Zeldalike lockandkey topology first (graph routing + gating logic)

 - Graph routing and gating logic (keys, locks, dependencies)

 - Dungeon “skinning” with templates (geometry + room placement)

 - Progression pacing & narrative beat spacing (discover’obtain’unlock without frustrating backtracking)

 - Pathfinding robustness against grid/route failures (avoid “lost”, “wrong travel”, “no path” cases)

 - Gameindependent evaluation framework (progression/readability/solvability metrics + benchmark loop)

 - Key/lock solvability as a graph constraint problem

 - Fitness function design for lock progression consistency & pacing

 - Gameindependent readability & progression metrics framework

 - Contractstyle evaluation checklist & robustness tests (progression satisfaction + pacingsignal verification)

Antioverfitting constraints for PCG Zeldalike player experience

- Meaning of the level model & interpreting gaps/questions in the PCG output

Evaluation metrics tied to traversal/pathfinding playability

Graph-grid pathfinding integration: constraints and how they affect PCG output

Zeldalike pacing constraints (tempo, spacing, backtracking loops)

Navigation coherence & criticalpath discovery rate

Constraint taxonomy for controllability (reachability, progression, required elements, navigability)

Fully automatic evaluation pipeline (no human study)

Joint optimization framing (layout + constraints + pacing)

BSP/regeneration layer as constraintsatisfaction bridge (adjacency/region graphs)

Research gaps in Zeldalike PCG evaluation (toward publicationready metrics)

Zelda-like PCG pipeline in Unreal Engine

PCG system overview (Zeldalike goals)

Unreal Engine integration (editor vs cook time vs runtime)

Evolutionary algorithms for Zeldastyle level generation

Combat and encounter pacing in procedural dungeons

Graph/feature pipeline structure for Zelda-style spatial rules

Preserving hub ' region ' revisit loops with deterministic seeds

Zelda dungeon grammar as hard designerauthored constraints (pacing, readability, setpieces)

Validator pipeline: solvability, traversal clarity, and flow checks

Feature/graph pipeline structure for Zeldastyle spatial constraints

Generator degrees of freedom: geometry variation within fixed grammar

PCG pipeline stages: topology/geometry generation vs validation gates

Adjacency graph extraction and preservation across generation steps

Solvability and playerunderstandable flow validation (navmesh reachability, deadlock checks, loop invariants)

Puzzleroom type generation (algorithms/ML options integrated with topo + pathfinding)

Topologyaware ML objectives to preserve graph characteristics (persistent homology/persistence images, loop persistence regularization)

Topologyaware training objectives for graph structure preservation

TopologyPreserving Constraints in GraphBased Level Generation
(citation: 1, 2)

TopologyAware Training Losses for Graph Structural Fidelity (citation: 1, 2)

Persistent homology / persistence image loss over traversal cost filter functions

Pathfinding robustness in PCG dungeons (walkable paths, links between regions)

Validator timing and stages

Deterministic PCG data model for Zelda constraints (locks/keys, tags/affordances, region budgets)

BSP collisionfree partition stage (constraintdriven room/corridor placement, iteration sandbox)

“PlayercanreachwhatIthink” solvability validation (Astyle checks using same nav/cost model)

Astyle reachability constraints parity (locks, oneway/forbidden, widths)

Landmarktolandmark thresholding (path cost + corridor count)

Representationmismatch safeguards in pathfinding (tilestate normalization, start/goal passability, coordinate/state pruning)

Adversarial and solverconsistency validation (boundary clamping, multi-heuristics, bidirectional A agreement, postroute simulation)

Tilestate normalization & grid convention alignment

Start/goal passability & edgecase gating

Fallback/repair validation before rejecting candidates

Graphdrift and progression integrity checks (room–corridor connectivity, locks/keys feasibility, chokepoints, biconnected regions, loops/revisit)

Requiredelement + navigation twocheckpoint enforcement (dependency graph reachability ' routereadability/learnability metrics)

Zeldaaligned scope & pacing validators (overworld'dungeon'revisit loop coherence, global progression beats)

Zeldastyle level generation workflow (PCG)

PCG navigation and layout design

spritelike dungeon/tile layout generation

Mission/quest graph ' room generation pipeline

Dungeon graph connectivity & room placement rules

Pathfinding constraints & topological characteristics

Binary Space Partitioning (BSP) for level/space partitioning (why use it)

VGLC dataset usage for 2D dungeon generation (formats & gaps)

Procedural puzzle room generation (algorithms / ML integration)

Puzzle room archetypes (room types, entrances/exits)

Topology + pathfinding integration (topo checks, connectivity, reachability)

Research gap identification & evaluation strategy for Zeldastyle PCG

Encoding and validating “pacing signals” across seeds

Paper submission and peer review

Check <https://storm.genie.stanford.edu/conversation/139529> for more details

Stanford University Open Virtual Assistant Lab

The generated report can make mistakes.

Please consider checking important information.

The generated content does not represent the developer's viewpoint.

Procedural content generation in video games (background)

Procedural content generation (PCG) in games: definitions and scope

Procedural content generation (PCG), also known as procedural content generation, is the creation of game data or content using algorithms rather than manually crafting it.^{[1][4][5]} In video games, it is commonly used to automatically (or computer-assisted) generate large amounts of content, enabling developers to reduce the cost of game design and development while increasing the variety of what players encounter.^{[2][3][5]} PCG may involve generating levels, landscapes, items, rules, and quests, among other elements of gameplay.^[9]

More broadly, PCG is described as the algorithmic creation of game content with limited or indirect user involvement, or as the use of algorithms and computational techniques to automatically generate data or content in games.^{[6][4]} Definitions also extend to cases where AI systems help author game aspects that would typically be handled by human designers, positioning PCG as a spectrum that can range from traditional rule-based automation to more advanced AI-assisted approaches.^[12]

Historically, PCG has been studied and used in the game industry as a technique for overcoming resource and time constraints, and it has continued to evolve alongside developments in deep learning and other computational methods.^[14] Recent work also highlights how generative AI can accelerate the creation of diverse game elements, potentially enhancing PCG while raising considerations related to creative ownership and originality.^[8] As a result, PCG is often discussed not only in terms of automated generation, but also in terms of its expanding scope—from deterministic content generation to increasingly dynamic, AI-enhanced systems.^{[14][15]}

In practical terms, PCG frameworks and tools support the creation of procedural content workflows. For example, the Procedural Content Generation Framework provides tooling for creating procedural content and related tools within game development environments such as Unreal Engine.^[10]

Zeldalike end-to-end world generation: scientific motivation and relevance

Zelda-like end-to-end world generation has been proposed as a way to address limitations of traditional procedural content generation workflows, where different stages—such as layout construction, asset placement, and rule- or puzzle-specific constraints—are often handled by separate or hand-designed components. In contrast, “end-to-end generation” refers to an approach in which a single integrated system performs the full generation process in one step, aiming to learn coherent world structure and gameplay-relevant features jointly rather than composing them from separately optimized modules.[\[178\]](#) This framing is relevant to procedurally generated Zelda-like experiences because such games require tightly coupled relationships between spatial structure (e.g., traversal paths and room connectivity), authored-like content (e.g., obstacles and puzzle elements), and player experience.

Recent work on end-to-end architectures has argued that modern deep learning methods can learn these kinds of high-level dependencies, but also notes that current systems often rely on large datasets and substantial computational resources.[\[179\]](#) This dependence is one reason the scientific motivation is sometimes discussed in terms of scalability and the ability to leverage available benchmark environments. For example, research on observational overfitting and related evaluation challenges highlights that learning-based generation systems must be assessed carefully to ensure that they are not merely reproducing correlations from training data.[\[175\]](#) In this context, procedural generation can be used to increase diversity and coverage of environments, potentially helping to mitigate memorization or overfitting to a narrow distribution.[\[176\]](#)

Empirically, end-to-end or deeply integrated generation approaches have been demonstrated in procedurally generated, puzzle-oriented 3D environments. “Obstacle Tower” is one example of a high visual fidelity, third-person procedural environment intended to produce nontrivial levels and puzzles, illustrating how procedural generation can target gameplay complexity rather than only aesthetic variation.[\[177\]](#) While such systems may not specifically replicate the full Zelda experience at production scale, they provide a research basis for studying how unified models or closely integrated pipelines can generate structurally meaningful levels that align with playability constraints.

Evolutionary algorithms for PCG game levels: typical representations and operators

Evolutionary algorithms are a class of population-based search methods commonly applied to procedural content generation (PCG), including game level design. In this approach, a set (population) of candidate content instances—such as level layouts, enemy placement patterns, or path configurations—is maintained in memory at the same time. Over successive iterations (generations), candidate levels are evaluated with a fitness function, and new candidate levels are produced by applying

evolutionary variation operators, such as recombination (crossover) and randomized mutation.[\[345\]](#)

Within PCG, typical representations convert a level into a format that can be manipulated by the evolutionary operators. Common representations include fixed-length vectors encoding tiles or components, graph- or grid-based encodings capturing connectivity between rooms or pathways, and parameterized descriptions that summarize structural features (e.g., number and size of rooms, placement constraints, or progression ordering). The choice of representation largely determines what kinds of alterations crossover and mutation can make while still producing “valid” or playable levels.

Typical operators are designed to explore the space of possible levels while preserving desirable properties. At a high level, evolutionary algorithms operate in a loop: (1) evaluate each candidate level with a fitness measure reflecting design goals (such as playability, difficulty targets, solvability, or aesthetic criteria), (2) select higher-fitness candidates to influence the next generation, and (3) generate offspring by recombining segments of parent representations and applying mutations that introduce variation. Because the underlying method relies on iterating on generations of populations, these representations and operators are central to balancing exploration (trying new structures) and exploitation (refining solutions that already satisfy constraints).[\[345\]](#)

Identifying research gaps and defining research questions for endtoend systems

Literature review strategy for procedural content generation and level design

A literature review on procedural content generation (PCG) for game levels and level design commonly begins by examining how established level design principles, such as critical path planning and risk-for-reward, can inform or constrain PCG algorithms.[\[1\]](#) These principles are often discussed in relation to “Zelda-like” structures, where curated progression and player agency are central, and where procedural methods are used not merely to generate content, but to preserve recognizable traversal and challenge patterns.[\[2\]\[3\]](#) Some work also emphasizes that PCG systems can be designed to support uncertainty and replayability, encouraging players to learn the underlying mechanics through repeated encounters with newly generated layouts.[\[4\]](#)

In reviewing PCG approaches, researchers frequently categorize methods by how they define player progression and content dependencies. One common framing uses lock-and-key structures to specify allowable paths and gating relationships for progression, which has been applied in roguelike contexts and can be generalized to broader procedural generation tasks beyond dungeon layouts.[\[5\]](#) Related discussions in the literature distinguish “unexplored” level design patterns that align more closely with 2D Zelda-style navigation than with purely random wandering,

highlighting the role of quickly revealing paths and guided exploration in procedural dungeons.[\[6\]](#)

Methodologically, the literature review strategy often surveys a range of procedural generation techniques used in game level and dungeon contexts. For example, one strand of research reviews multiple procedural methods—such as midpoint displacement, random walks, cellular automata, and Perlin noise—within the broader landscape of procedural generation approaches relevant to level design.[\[7\]](#) Other studies focus more specifically on empirical or agent-based generation pipelines, including the use of reinforcement learning (RL) agents to generate level layouts for 2D-style dungeon crawler games explicitly modeled after Zelda-like experiences.[\[8\]](#) Overviews of PCG in games are also used as grounding material, clarifying core definitions such as generating content “from scratch” either before gameplay or during runtime (including examples like loot generation) and establishing terminology used throughout the level-design-focused literature.[\[9\]\[10\]](#)

Translating gaps into precise research questions for end-to-end systems

In end-to-end procedural content generation (PCG) pipelines—where algorithms generate not only levels but also related gameplay elements such as quests—research gaps can become difficult to articulate without a structured approach. PCG is commonly described as the automatic, algorithmic creation of game content rather than manual authoring by designers.[\[1\]\[2\]\[4\]](#) While this capability supports goals such as producing connected and impactful dynamic quests in open-world games, it also complicates the identification of what, specifically, is failing and why (e.g., which part of the pipeline breaks narrative coherence, difficulty progression, or player goals linkage).[\[3\]](#) As a result, translating broad “gaps” into precise research questions is necessary to guide evaluation and improvement efforts.

A common starting point is to convert informal or systemic concerns into testable objectives tied to specific algorithmic components. For example, research that evaluates “strengths and weaknesses” of procedural generation algorithms for 2D games can be reframed into questions about comparative performance and failure modes across algorithm classes.[\[5\]](#) Similarly, surveys and research summaries that define PCG and characterize its state often motivate “gap identification” work—explicitly aiming to pinpoint limitations in existing approaches and to motivate remedies.[\[6\]](#) Doing this for end-to-end systems requires asking questions at the level of controllability, since designers typically need predictable influence over outputs rather than opaque generation.[\[7\]](#)

One practical translation method is to define research questions around the degree and form of designer control over PCG outputs. Taxonomies have been proposed specifically to focus on “the control available to a designer,” including both the type of control and the extent to which it can be exercised.[\[7\]](#) For end-to-end systems, this translates into questions such as: How can constraints, rules, or designer-specified parameters propagate through the full pipeline to yield coherent downstream content (e.g., quests aligned with generated spaces)? Evidence from PCG research direc-

tions that introduce constraint- or rule-driven methods suggests that designers can specify sets of rules and constraints, after which the system generates content consistent with them.[\[8\]](#) This yields more precise questions about constraint satisfaction, robustness under competing objectives, and the reliability of cross-stage consistency.

Another translation method is to connect gaps to end-user experience outcomes and production trade-offs. Research on procedural level generation highlights the potential to provide engaging player experiences while potentially reducing development effort,[\[9\]](#) which can be translated into questions about measurable player-facing quality metrics (engagement, perceived connectedness, or replay value) and measurable production metrics (iteration time, cost, or manual cleanup rates). For dynamic quest systems, this can further become a question about how generated content remains meaningfully linked rather than merely “plausibly” related, reflecting the broader aim that generated worlds can be reassembled with new meaning from existing semantics.[\[10\]](#)

Finally, it can help to formulate research questions that explicitly address the gap between idealized generation goals and practical end-to-end implementation. Since PCG is often framed as computer-generated content executed via well-defined algorithmic processes,[\[11\]](#) research questions should ask what additional mechanisms are required in full systems to handle dependencies among multiple generated artifacts. In this framing, gaps are not only “algorithmic quality” issues but also “system integration” issues: how to ensure that upstream generation outputs (maps, regions, or other structural semantics) lead to downstream content (quests, objectives, or narrative events) that is consistently compatible with player expectations and game rules across the whole pipeline.[\[3\]\[9\]](#)

PCG game level design: literature to prototype workflow (papers, tutorials, next steps)

A typical workflow for designing game levels with procedural content generation (PCG) combines research on established level-design principles with iterative prototyping and tool support. Early guidance often focuses on practical implementation steps, such as creating and managing map objects and then applying placement algorithms to populate levels automatically.[\[283\]](#) Complementary tutorial materials describe “handy helper-methods” and the data structures needed to implement random-level generation, emphasizing that working systems usually require more than just a single generation function.[\[284\]](#)

Researchers and practitioners also connect PCG algorithms to traditional level-design concepts—for example, using ideas such as a critical path and risk-for-reward to shape procedural outcomes rather than producing purely random layouts.[\[285\]](#) In this view, PCG is not only a means of generating variety, but is also used to increase replay value, support data compression and data reuse, and enable greater scalability in game content pipelines.[\[286\]](#) Conceptually, procedural content generation is commonly defined as using an AI system to author aspects of a game that a human designer would typically be responsible for, which frames level generation as an automated design process rather than a purely technical one.[\[287\]](#)

In terms of practical tooling, modern workflows may leverage engine-specific frameworks to accelerate prototyping. For example, the Procedural Content Generation Framework (PCG) is described as a toolset for creating procedural content and corresponding tools inside Unreal Engine, enabling developers to experiment with generation logic directly within a production environment.[\[288\]](#) Together, these resources support a literature-to-prototype loop: study how existing design principles can constrain and guide generation, prototype placement and data-structure strategies, then validate results using engine-integrated PCG tooling before moving toward more advanced algorithms and production-ready pipelines.[\[283\]](#)[\[284\]](#)[\[285\]](#)[\[288\]](#)

Zelda-like PCG: Key constraints and evaluation metrics

Key/lock progression modeling for guaranteed solvability

Zeldalike lockandkey topology first (graph routing + gating logic)

Graph routing and gating logic (keys, locks, dependencies)

Graph routing and gating logic in Zelda-like procedural content generation (PCG) refers to the use of an explicit dependency structure—often represented as a graph—to determine how progress-restricting elements such as keys, locks, and switches control traversal. At its core, the lock-and-key model focuses on specifying paths of progression along with constraints that regulate which parts of the graph become available when the player obtains the necessary items or triggers.[\[412\]](#)

A common approach begins by analyzing or constructing a mission/room graph that captures multiple possible routes through the level, enabling the designer or generator to reason about route complexity and difficulty before committing to detailed layouts.[\[413\]](#) One method for producing the underlying connectivity is to build a room adjacency graph from spatial relationships—for example, by constructing a graph of room center points using Delaunay triangulation so that rooms become connected in a coherent navigation network.[\[414\]](#) From this connectivity, the generator determines progression ordering by enforcing acyclic or otherwise well-structured constraints on how gated regions can be reached.

To guarantee that generated dungeons are solvable while still meeting design constraints, graph-based and constraint-solving techniques can be used. Research on controllable graph-based models of acyclic dungeon levels describes building such structures via declarative constraint solving.[\[415\]](#) Similarly, placement steps can be performed by formulating requirements as integer linear constraints and solving them with satisfiability modulo theories (SMT), which supports reasoning about dependencies between keys, locks, and reachable regions.[\[416\]](#)

After the dependency-aware topology is established, the model can be “skinned” by mapping the graph into the game’s spatial representation, including generating

room and hallway geometry consistent with the routing and gating rules.[\[417\]](#) In practice, this means that gating logic is treated as a first-class design artifact: keys are placed and locks are assigned so that the intended progression paths exist and remain reachable, while unreachable states are avoided. For example, a generator may first create a fair, single-solution traversal structure for a puzzle-like subspace and then layer route-gating dependencies—such that a path segment becomes traversable only after a corresponding key-switch elsewhere is activated—while validating solvable path length and backtracking cost using the same underlying navigation/cost model used for repair and verification.[\[507\]\[508\]\[509\]](#) Perfect-maze techniques such as depth-first search backtracking (including its randomized variants) and minimum-spanning-tree approaches like Kruskal’s or Prim’s can provide the single-solution “fairness” baseline that the lock-and-key gating logic then constrains.[\[508\]\[509\]\[510\]](#)

Dungeon “skinning” with templates (geometry + room placement)

After a lock-and-key dungeon’s underlying progression topology has been established, the next step is “skinning” the abstract layout with concrete geometry and reusable room templates. This stage turns the purely graph-based plan into a play-space that resembles the pacing and readable traversal of early dungeons, while keeping the guaranteed solution intact across seeds.[\[406\]\[411\]](#) In practice, developers can treat room placement and spatial composition as a constrained mapping problem: each node in the progression graph is instantiated as a room template, and each corridor/edge is realized with geometry rules that preserve reachability, line-of-sight clarity, and navigation consistency.[\[406\]\[407\]](#)

Template selection and placement can also be guided by gameplay-oriented validation metrics. For example, after instantiating the dungeon shell, the generated result can be evaluated for “designed-feel” signals such as line-of-sight readability (so players can infer where to go next), combat-and-traversal rhythm (so encounters and movement segments align with expected pacing), and reachable treasure placement (ensuring key rewards remain obtainable without bypassing intended gating).[\[406\]](#) Because the lock-and-key structure can be decoupled from the level’s visual and spatial realization, the same guaranteed progression topology can be reused while varying the dungeon’s appearance through different room and corridor templates, supporting consistent comprehension even as layout elements change between runs.[\[407\]\[408\]](#)

This approach can be applied beyond classic Zelda-style key-and-lock puzzles, including the generation of other content types such as branching dialog, by reusing the same decoupled “topology first, skin later” workflow.[\[408\]](#) For dungeon-specific work, a common implementation is to focus on dungeon plans that use only normal keys (i.e., no additional key items or breakable-wall mechanics), then render them into full playable spaces using validated templates to maintain the intended progression order.[\[409\]](#) Related research has also framed dungeon verification as a heuristic problem for ensuring that procedurally generated, graph-based dungeon structures satisfy constraints implied by Zelda-like key-and-lock challenge ordering.[\[410\]\[411\]](#)

Progression pacing & narrative beat spacing (discover'obtain'unlock without frustrating backtracking)

Procedurally generated Zelda-like levels often rely on pacing mechanics that mirror the classic gameplay loop: players first discover an area, then obtain a dungeon-relevant item, and finally use that item to unlock further regions or routes. Designing for this “discover ’ obtain item ’ unlock new area” narrative beat spacing requires aligning the level's progression structure with the key/lock gating so that new capabilities are introduced in an order that matches the spatial layout being revealed. In practice, this means treating the dungeon and its progression barriers as a coupled system rather than independent map elements—ensuring that the player's traversal naturally leads to the next item acquisition, and that returning to previously explored spaces is either unnecessary or limited to intended, non-frustrating paths.[\[44\]](#)

A core design goal is solvable-by-construction progression, since pacing collapses if players can discover locks early but cannot obtain the required items until later. Dungeon generation approaches therefore focus on properties like reliability and solvability, evaluating whether a generator consistently produces levels in which progression is achievable and where gating behaves as intended across runs.[\[45\]](#)[\[46\]](#) One solvability strategy is to systematically structure key placement relative to the entrance so that locked regions are encountered only after the corresponding keys could plausibly be collected; for example, moving keys one room toward the entrance (or otherwise enforcing monotonic key-to-entrance ordering) can help guarantee that every region remains reachable under the intended sequence, supporting smooth progression beats without dead ends.[\[47\]](#) In contrast, some puzzle-generation patterns reduce the system to simpler “one lock, one key” dynamics, which can aid solvable pacing but may limit expressivity in how richly paced discoveries and item-based unlocking are interleaved across a map.[\[48\]](#)

Beyond correctness, beat spacing depends on how frequently the player's route alternates between exploration and payoff. Constraint-aware procedural generation systems aim to produce diverse levels while meeting progression constraints, enabling consistent narrative pacing across different layouts rather than relying on handcrafted handoffs.[\[51\]](#) Evaluation methodologies for dungeon generators similarly connect structural measures (such as connectivity and solvability) to practical level outcomes, supporting the idea that pacing can be treated as an emergent property of the generated structure and its gating rules.[\[46\]](#) Research into evolutionary dungeon generation with locked-door missions further reflects this perspective: generating progression content is not only about placing locks, but about ensuring that the resulting experience supports a coherent run of discoveries, item acquisition, and subsequent unlocking.[\[52\]](#)

Pathfinding robustness against grid/route failures (avoid “lost”, “wrong travel”, “no path” cases)

Pathfinding robustness is important for procedural level generation in Zelda-like games because level validity and subsequent content evaluation often depend on

whether the generated navigation graph yields reliable routes. In practice, pathfinding agents may fail in multiple ways—such as getting “lost” on a grid, taking an incorrect route, or returning “no path” even when a solution should exist—leading to downstream issues in tasks like graph generation, difficulty evaluation, and route gating.[\[264\]](#)

Common failure modes occur at map boundaries or during tile-state sampling. For example, when pathfinding logic checks whether an edge tile is passable, it may encounter undefined tiles at the borders; if these are treated as non-passable, the algorithm can incorrectly block traversal and fail to find routes.[\[265\]](#)

Robust procedural layouts often mitigate these issues by validating connectivity after key placements and other structural modifications. One approach is to randomly select a room location for an item (such as a key), then explicitly test whether a valid path still exists between the player’s start and the route leading to the key; if no path exists, the placement is rejected or regenerated.[\[266\]](#)

Some implementations also need to account for algorithm behavior across multiple calls or iterations. Reported inconsistencies can stem from how previously visited coordinates are tracked or reset, causing repeated calls to behave unexpectedly and produce errors such as revisiting states incorrectly.[\[267\]](#) Additionally, some “multiple path” failures may be limited to the scope of a single pathfinding invocation, while other calls still behave correctly—suggesting that state management (e.g., visited sets, caches, or scoring) should be carefully isolated per call.[\[268\]](#)

Procedural environments also tend to differ from handcrafted maps with pre-baked navigation data. Developers have noted that AI pathfinding may work correctly on non-randomized maps where navigation meshes are baked, but fail in procedural settings, requiring adjustments to ensure the runtime navigation representation matches the generated geometry.[\[269\]](#)

Finally, because robust validation may require multiple expensive path queries during generation, performance constraints should be addressed. Since pathfinding can freeze gameplay when executed synchronously, a common mitigation is to split path computations across time steps (or otherwise amortize work) while still ensuring that connectivity constraints and route correctness are maintained.[\[270\]](#)

To support gameplay goals like solvability guarantees and correct lock-and-key progression, robust pathfinding validation is typically paired with maze or route generation techniques that preserve well-defined solutions (e.g., using “perfect” mazes with unique paths) and then enforcing route dependencies only when the underlying path model confirms reachability.[\[501\]\[503\]\[504\]\[505\]](#)

Gameindependent evaluation framework (progression/readability/solvability metrics + benchmark loop)

Key/lock solvability as a graph constraint problem

In games inspired by , procedural level generation often aims to ensure that key progression is possible in a specific order (e.g., locked areas become accessible only

after obtaining the appropriate keys or activating required elements). One approach to formalizing this requirement is to model the level as a graph constraint problem in which non-spatial dependencies—such as keys, codes, switches, and other activations—are resolved before spatial placement. This supports the generation of “solvable & progressive” dungeon layouts by separating dependency planning from later spatial constraint satisfaction.[\[237\]](#)

In such graph-based formalisms, the level is represented as a stateful progression structure that encodes progression constraints relative to the current state of the player and the presence or absence of required items. Solvability is then enforced through constraint solving that guarantees reachability from a start configuration to intended objectives while respecting lock/key ordering requirements.[\[236\]](#) The constraint model can explicitly target criteria including (1) start-to-goal reachability through the intended action set, (2) critical-path solvability under lock/key dependencies, (3) functional activation of required elements (i.e., that specific triggers unlock the progression actions they are meant to enable), and (4) navigability and route diversity (i.e., avoiding dead or unreachable pockets and allowing multiple traversal options). These checks are typically applied in staged phases of procedural generation to validate that the produced layout remains playable as dependencies are instantiated.[\[232\]](#)

Related work on constraint-based and graph-based procedural generation further motivates this view by treating solvability as a property that can be ensured via graph-encoded progression constraints rather than by ad hoc validation. For example, level-generation systems using constraints describe progression solvability as something ensured through the structure of the encoded graph, where constraints account for the dependencies implied by locks and other gating mechanisms.[\[236\]](#) Additionally, graph-based procedural algorithms, including those employing declarative constraint solving, have been used to build controllable models of dungeon-like levels in ways that support solvability considerations as part of the underlying representation.[\[240\]](#)

Fitness function design for lock progression consistency & pacing

In graph-based procedural dungeon generation, the fitness function is central to enforcing and to shaping throughout a level. One approach specifies a dungeon genotype as a graph whose structure encodes progression dependencies—such as rooms connected by doors, key locations, and terminal nodes for a boss or exit—and then evolves candidate graphs using evolutionary operators that preserve reachability constraints.[\[346\]\[348\]](#) Under this formulation, the fitness function evaluates whether the generated layout remains solvable while respecting lock-and-key ordering, including constraints on which locked doors can be reached at each stage of the run.[\[346\]\[350\]](#)

Progression consistency can be measured by scoring how well the dungeon’s reachable subgraphs evolve as keys are collected. This includes checking that door placement aligns with key availability along intended paths, so that unlocking events occur in a feasible order rather than producing dead ends or “unwinnable until late” regions.[\[346\]\[350\]](#) For multi-lock and multi-key designs, the fitness function

can further incorporate the presence of optional routes and the overall “feel” of the puzzle structure by assessing how different lock chains interact across the mission graph.[\[351\]](#) In addition, candidate layouts may be penalized when they appear solvable under an automated test harness but are deliberately structured in a way that would be unfriendly or unintuitive for players, ensuring that the algorithm’s notion of solvability corresponds to intended play experience.[\[346\]](#)

Pacing is typically handled by extending these solvability checks with timing- or progression-depth-aware terms. One method uses an explicit accessibility model in which each node in a mission graph is assigned an access level representing how many puzzle/lock nodes must be passed before reaching it, enabling the fitness function to prefer progression patterns that distribute challenges across the play session rather than clustering them early or late.[\[357\]](#) Complementary verification heuristics can also be incorporated to validate procedurally generated graph dungeons, treating the generation problem as a constrained search and using constraint-based verification to support a fitness signal focused on correctness of the puzzle’s progression constraints.[\[352\]](#) Such designs align the evolutionary objective with the lock-and-key model’s core idea: specifying not only which paths exist, but also the constraints governing progression along those paths.[\[350\]](#)

Gameindependent readability & progression metrics framework

Procedural generation is commonly defined as techniques that create data algorithmically rather than manually.[\[128\]](#) In game contexts, it is often used to increase replay value, compress and reuse data, and improve scalability.[\[127\]](#) However, evaluating the quality of procedurally generated game levels—especially for Zelda-like progression structures—can be difficult when relying on game-specific design intuition or subjective “feel.”[\[126\]](#)

To address this, a game-independent evaluation framework can be grounded in measurable properties of solvability, progression, and readability that can be computed across different generators and seeds. One such approach is to treat a level’s structure as a dependency graph and evaluate how effectively players can discover and navigate critical paths through that graph.[\[126\]](#) Readability metrics can then focus on navigation coherence (for example, the rate at which the critical path is discovered during play or simulation), while progression metrics can quantify consistency in key–lock or gate-based advancement patterns.[\[126\]](#) In addition, pacing-oriented measures can be expressed as cadence signatures—such as the timing and sequencing of different room or encounter types—to capture whether generated content matches the expected rhythm of progression.[\[126\]](#) Together, these measures support an optimization and benchmarking loop that targets progression comprehension and navigation clarity, rather than only generic notions of playability.[\[126\]](#)

Contractstyle evaluation checklist & robustness tests (progression satisfaction + pacingsignal verification)

A contract-style evaluation checklist for Zelda-like procedural dungeon generation can be organized around and , treating each dungeon as a constrained system of

progression states and temporal “beats” rather than as only a layout problem.[\[91\]](#) Under a lock-and-key model, the core requirement is that the generator specifies progression paths while respecting the constraints that gate those paths (e.g., which barriers are temporarily blocking progress and which keys unblock them).[\[91\]\[96\]](#) To validate this, designers can define—at minimum—(1) satisfiability of key/lock dependencies and (2) correctness of the player’s pacing signals across hub’region’revisit loops.[\[91\]\[92\]](#)

Progression satisfaction (key/lock + boss-key gating). The checklist should verify that progression is possible and unambiguous under the intended key usage semantics. For example, a common constraint is that locks/doors can be opened only under specific conditions (such as “each key can only be used once” and doors remain open after being unlocked), which must be reflected in the state model used to test candidate dungeons.[\[93\]](#) Locks should be interpreted as barriers that stop further progress and keys as features that enable that progress, so the evaluation can be cast as a reachability problem over progression states.[\[94\]](#) Practical tests include: (a) reachability from dungeon entry to the boss under the generated key/lock rules; (b) correctness of any boss-key gating such that the boss encounter is effectively sequenced after acquiring the appropriate access item; and (c) dependency auditing to ensure that no lock requires a key that cannot be obtained prior to the lock (and, conversely, that no obtained key is rendered permanently irrelevant by its dependency chain).[\[94\]\[96\]](#) Because lock-and-key procedural generation is often framed as letting designers author and control constraints “expressing gameplay-related design constraints,” the checklist should explicitly correspond its tests to the constraints given to the generator.[\[95\]](#) If the dungeon includes temporary barriers, the placement and assignment of barriers and keys must be checked so the progression “unlocks” exactly when intended by the contract.[\[96\]](#) Visual/UX conventions can also be treated as part of progression satisfaction—e.g., ensuring that icons/affordances for locked doors and boss-key sections appear in the expected places so players can consistently infer required actions.[\[99\]](#)

Pacing-signal verification (hub’region’revisit beat spacing). In addition to solvability, Zelda-like dungeons rely on readable pacing signals that cue the player’s movement through a looped structure (such as hub’region exploration and later revisits). An evaluation checklist should therefore verify that the generated dungeon produces the intended sequence of discovery, escalation, and backtracking opportunities by measuring spacing between progression-relevant “beats” (e.g., when new items become testable, when the path tightens toward the boss, and when revisits are meaningfully productive rather than merely repetitive).[\[91\]\[104\]](#) A robust pacing test can combine structural assertions (e.g., that each region is accessible in a way that supports planned revisit beats) with traversal metrics (e.g., whether the player is repeatedly funneled into regions that fail to create new options after a key pickup). This mirrors the idea that Zelda dungeons often work best as self-contained spaces, where restricting a small key to the dungeon where it is found helps ensure that the player’s pacing signals stay consistent within the dungeon’s own loop.[\[102\]](#) Moreover, designers can use known Zelda design patterns to define evaluation targets for timing and linearity versus atmosphere; for instance, observations about puzzle motif

introduction and subsequent shifts in Zelda dungeon linearity can be used to form expectations about how “signals” should feel even when layouts vary.[\[103\]](#) This can be operationalized by checking that after key acquisition, the player encounters a timely set of interactable locks that allow the new item to be tested before the final challenge, aligning with the expectation that item usefulness is validated in the lead-up to the boss.[\[104\]](#)[\[101\]](#)

Robustness tests (end-to-end guarantees). Beyond correctness on individual generated instances, robustness tests should probe the generator’s behavior under variation in constraints, random seeds, and player/solve-order assumptions. One axis is of the dependency model: if the system assumes doors open permanently and keys may have limited/one-time use, then the evaluation should test that the dungeon remains solvable across the full range of possible state transitions, not just a single plan.[\[93\]](#) Another axis is : because lock-and-key generation can be framed as authoring constraints, robustness tests should include adversarial checks that attempt to violate or nearly violate contract conditions (e.g., keys placed too late, barriers released too early, or boss-key dependencies that become circular), ensuring the generator either rejects the candidate or regenerates until the constraints are satisfied.[\[95\]](#)[\[96\]](#) Finally, should test whether hub’region’revisit signals remain readable under different traversal choices, verifying that the temporal structure still produces a coherent escalation toward the boss rather than undermining it through overly long dead phases or premature access that breaks the intended beat structure.[\[91\]](#)[\[92\]](#)

Antioverfitting constraints for PCG Zeldalike player experience

Meaning of the level model & interpreting gaps/questions in the PCG output

Evaluation metrics tied to traversal/pathfinding playability

In procedural level design, evaluation often focuses on how playable the generated spaces are when traversed by a character or agent using pathfinding. This approach links algorithmic outcomes—such as reachability between points, traversal cost, and obstacle navigation—to the perceived quality of a level’s layout, rather than relying solely on structural properties of the generated content.[\[295\]](#) Accordingly, evaluation methods may treat the level as a grid or spatial graph and assess whether the underlying navigation model can move through the map under the constraints implied by the generated geometry and connections.[\[295\]](#)[\[296\]](#)

For example, metrics can be derived from the agent’s ability to find valid routes across the level while respecting both graph connectivity and spatial layout (e.g., terrain as grid cells combined with graph nodes/edges for movement decisions), yielding playability measures tied directly to traversal behavior.[\[295\]](#) When used to interpret or “fill gaps” in a level-generation model, such traversal-centric evaluations help clarify what the model is optimizing—namely, producing layouts that are navigable,

goal-reachable, and sufficiently traversable under the same pathfinding assumptions used during evaluation.[\[295\]](#)[\[296\]](#)

More broadly, procedural content generation research emphasizes difficulty and control in ways that can be quantified through navigation and constraint satisfaction, including evaluation pipelines that incorporate procedural generation with difficulty regulation and language- or model-conditioned design intents.[\[297\]](#) In this context, traversal/pathfinding metrics serve as a practical bridge between generated level structure and user-relevant gameplay outcomes, enabling automated evaluation of whether a level supports coherent movement and challenge consistent with the generation goals.[\[295\]](#)[\[297\]](#)

Graph–grid pathfinding integration: constraints and how they affect PCG output

Graph–grid pathfinding integration is a common design strategy in procedural content generation (PCG) for games, where layout generation is constrained by traversability and intended player movement. In this context, developers often combine grid-based representations (for local movement and collision checks) with graph-based representations (for higher-level connectivity between regions or nodes). The resulting pathfinding evaluation helps ensure that generated spaces form coherent routes through the map and that gameplay-relevant traversal is possible.[\[298\]](#)[\[299\]](#)

PCG is generally understood as the algorithmic creation of game content (such as environments, levels, and even placements of interactive elements) with minimal human input.[\[298\]](#)[\[301\]](#) When pathfinding is integrated into the generation loop, it effectively turns navigability into an explicit constraint: candidate level designs are produced and then tested by routing agents across the integrated graph–grid structure to verify that key areas are reachable and that traversal matches the intended structure of the level.[\[299\]](#) This constraint-based evaluation can also be used to support designs that “replicate, generalize, or extend” implicit design knowledge present in playable content, because the routing outcomes act as a proxy for whether the generated layout respects known layout patterns and connectivity expectations.[\[300\]](#)

Search- or evolution-based approaches further illustrate how path-related objectives can guide PCG output. For example, generative methods may evolve levels by targeting an intended sequence of player actions, which implicitly depends on navigable paths through the level’s geometry.[\[303\]](#) In such setups, the graph–grid integration helps translate raw generated geometry into movement-meaningful structures: the generator proposes a layout, the combined representation determines feasible routes, and the fitness or validity of that proposal depends on whether the agent can follow the required traversal sequence.

However, the use of algorithmic traversal constraints can also influence the style of generated content. One reported danger of procedural generation is producing overly uniform, predictable results—e.g., layouts that are “smooth” blends of similar patterns—because constraints may encourage repeated solutions that satisfy connectivity while limiting variety.[\[305\]](#) As a result, designers often need to balance strict traversability constraints with other objectives (such as exploration variety,

encounter pacing, or regional diversity) to avoid narrowing PCG output to a small set of graph–grid routable archetypes.[\[305\]](#)

Zeldalike pacing constraints (tempo, spacing, backtracking loops)

Zelda-like procedural level generation places emphasis not only on solvability, but also on pacing—the tempo at which the player encounters spaces, obstacles, and opportunities to act. One constraint is maintaining a consistent progression rhythm that supports “player intent,” including meaningful choices, rewarding exploration, and controlled backtracking rather than dead-end traversal.[\[16\]\[17\]](#) In this view, pacing can be evaluated alongside connectivity: dungeon generators are commonly assessed in terms of reliability (producing valid, playable dungeons consistently) and expressivity (generating levels that exhibit the intended structural variety and gameplay behaviors), rather than optimizing purely for whether a path exists.[\[18\]](#)

Beyond basic structural metrics (such as tile counts and connectivity/solvability), evaluations often include domain-specific measures tied to play feel, including playability, novelty, and robustness.[\[19\]\[20\]](#) For Zelda-like pacing, a key practical design constraint is guiding dungeon topology so that rooms become progressively reachable while still allowing the player to revisit earlier areas when new information or routes open up. For example, techniques that systematically move keys toward the entrance can ensure full reachability of rooms (and thus solvability) while preserving the opportunity for exploration-driven progression.[\[22\]](#) Similarly, incorporating lock-and-key structures provides a pacing mechanism: locked progress gates the player’s movement, while collected keys (and their placement/order) structure when and how backtracking becomes valuable.[\[24\]](#)

Because pacing is rarely captured by a single metric, some approaches treat evaluation as multi-objective, combining competing criteria to balance diversity with gameplay-relevant properties.[\[20\]\[21\]](#) Reinforcement learning–based multi-objective procedural generation has been proposed to train multiple generators toward diverse outputs while optimizing for multiple factors, which can include pacing-related behaviors in addition to validity.[\[21\]](#) Finally, systems intended to preserve player freedom can explicitly target the ability for players to “find a play style and pace that suits them,” suggesting that pacing constraints should accommodate different traversal and exploration strategies rather than enforcing a single script-like route.[\[25\]](#)

Navigation coherence & criticalpath discovery rate

Navigation coherence describes how consistently a player can reason about where to go next from the spatial and gameplay cues offered by a level. In Zelda-like design, this coherence is often evaluated by identifying the “critical path,” an idealized player flow that emphasizes the most important portions of the level and their dependencies.[\[120\]](#) The critical path is conceptually analogous to the Critical Path Method (CPM) used in scheduling: CPM finds the longest sequence of interdependent tasks, i.e., the path that constrains the earliest completion of a project.[\[121\]\[124\]](#) Applying this idea to level traversal supports metrics that quantify not only traversal feasibility, but also how reliably players can discover and follow the intended progression.

A practical way to operationalize this is through a “critical-path discovery rate,” measuring the proportion of playthroughs (or automated agent runs) in which the player discovers the critical path without requiring disproportionate backtracking or deviation. Because the critical path represents only part of overall activity, a balanced design target can help interpret discovery-rate results—for example, planning for roughly 10% to 20% of activities to lie on the critical path.[\[122\]](#) Under such a framing, navigation coherence improves when players are guided toward key decisions and dependencies often enough to maintain progression flow, while still allowing meaningful optional exploration off the main route via short offshoots from a primarily linear layout.[\[119\]](#) This makes the evaluation more amenable to procedural generation and comparisons across seeds, since coherence can be assessed in terms of discoverability of the critical route rather than subjective “feel.”[\[119\]](#)[\[120\]](#)

Constraint taxonomy for controllability (reachability, progression, required elements, navigability)

A constraint taxonomy for controllability in procedural content generation (PCG) can be organized around properties that determine whether a generated game level is navigable, learnable, and ultimately completable. Such constraints are commonly discussed in terms of [\[222\]](#)

Reachability constraints specify which regions, targets, or states are attainable from a given starting situation under the game’s mechanics. In tabular navigation settings, reachability is treated as the existence of a path to one of several equivalent targets, linking controllability to whether generated environments permit the player to get to intended locations.[\[223\]](#) More broadly, level design goals that involve navigation and puzzles depend on whether the player can traverse the environment in a way that supports the intended gameplay flow.[\[224\]](#)

Progression constraints capture how a player’s state changes as the level unfolds—often tied to advancing the story or quest through successful navigation. Levels are typically expected to allow players to move from earlier tasks toward later ones, such as by navigating through the environment to advance narrative goals.[\[225\]](#) In definitions of interactive storytelling components embedded in PCG-oriented design, progression can also depend on how scripted narrative segments and player actions unfold across the experience.[\[226\]](#)

Required elements constraints define which interactive components must appear and how they must connect to gameplay actions. For learning or teaching mechanics, required elements can include an initial starting state and an available action pair ((s,a)), implying that controllability depends on ensuring the level contains the informational and mechanical prerequisites for meaningful interaction.[\[223\]](#) At a higher level, PCG frameworks frequently distinguish mandatory story or gameplay elements from optional ones—for example, a “story script” as a required component within a PCG-informed design structure—emphasizing that the presence (and placement) of certain elements is necessary to avoid abrupt or non-communicative outcomes.[\[226\]](#)

Navigability constraints describe the structural and representational properties that make the level explorable and interpretable, including how players are able to find

routes and discover interactions. PCG discussions of navigation often consider how navigation tasks can be represented within interface structures (e.g., hypertext-like designs) and how player movement supports puzzle solving and story navigation.[\[224\]](#) Additionally, incorporating required elements for activating specific actions with useful outcomes can be treated as part of making the environment navigable in practice, since the player must be able not only to reach locations but also to find the appropriate affordances to proceed.[\[225\]](#)

Taken together, these constraint categories provide a practical basis for designing PCG levels whose controllability aligns with player expectations of learnability and completion: players must be able to reach intended targets (reachability), move through the level toward story/goal states (progression), encounter the necessary interactive/gameplay components (required elements), and interpret and traverse the space to find those components (navigability).[\[222\]\[225\]](#)

Fully automatic evaluation pipeline (no human study)

A fully automatic evaluation pipeline for procedurally generated game levels uses quantified heuristics and algorithmically defined fitness functions to score candidate levels without any human rating, playtesting, or interactive review.[\[204\]\[207\]](#) In this approach, a level generator produces a set of candidate content (e.g., chromosomes or generated layouts), and each candidate is evaluated by one or more automatically computed metrics that reflect desired properties such as quality, controllability, and diversity.[\[207\]\[205\]](#) The computed fitness values then guide selection and variation operators in optimization methods such as evolutionary algorithms (including approaches that adaptively weight fitness components), enabling the system to preferentially generate levels with higher predicted quality according to the metrics.[\[206\]\[205\]](#)

Because many PCG evaluation frameworks use both automated metrics and human evaluation, a purely automated pipeline deliberately omits any human-in-the-loop component and relies entirely on metric computation at evaluation time.[\[208\]](#) However, automated metrics—while scalable and efficient—may still fail to capture nuance, context, or other subjective aspects of play that human evaluators can perceive, motivating careful metric design and validation solely against automated criteria.[\[209\]](#)

In practice, the pipeline typically combines (1) a feature extraction stage that measures controllability/quality/diversity-related properties of each generated level, (2) a fitness function that aggregates these features into a single scalar score (or a multi-objective vector), and (3) an optimization or ranking step that selects or refines the highest-scoring levels for further generation.[\[207\]\[206\]](#) Metrics-based fitness functions are thus the core mechanism enabling full automation of level-quality evaluation in procedural content generation systems.[\[204\]\[205\]](#)

Joint optimization framing (layout + constraints + pacing)

Joint optimization framing is an approach to procedural dungeon and level generation that models multiple interdependent design dimensions—such as spatial layout,

rule-based constraints, and gameplay pacing—within a unified generation process rather than as separate downstream steps. In this view, the generator is treated as a system whose output must simultaneously satisfy structural requirements (e.g., connectivity and placement feasibility) while also producing sequences that achieve intended tempo and rhythmic progression during play.[\[181\]](#)[\[182\]](#)[\[186\]](#)

A common motivation for joint optimization is that coupling layout decisions with constraint satisfaction can reduce the brittleness of multi-stage pipelines, because layout changes can directly respond to unmet structural requirements. In purely constraint-based generation, for example, room layout can shift to satisfy constraints on level components, illustrating how the solver’s search over layouts can be driven by constraint satisfaction rather than fixed by an earlier handcrafted plan.[\[183\]](#)[\[188\]](#) Related declarative approaches emphasize that designers can specify intent and let the system search for layouts consistent with those requirements.[\[191\]](#)

To support constraint-driven placement and structure, some methods express placement as a set of formal constraints and solve them using satisfiability techniques. For instance, integer linear constraints have been solved using a satisfiability modulo theories (SMT) solver during the placement step, demonstrating how constraint programming can be used to ground generation in explicit, checkable rules.[\[189\]](#) In addition, constraint propagation frameworks may incorporate computed survivability or feasibility tests, which can be efficiently integrated into the system’s reasoning process rather than evaluated after-the-fact.[\[190\]](#)

Pacing and “rhythm & tempo” are often treated as an additional optimization objective layered onto these structural concerns. Conceptually, pacing can be linked to progression patterns (such as resource- or arcade-like stage advancement) so that generated content produces an intended experience of escalation and recovery across a play session.[\[184\]](#)[\[185\]](#) Extending this idea, research on dungeon representations has explicitly focused on repetition, rhythm, and connectivity, aligning pacing-related structure with the same representational backbone used for ensuring traversability.[\[186\]](#)

More broadly, “end-to-end” Zelda-like generation can be framed as learning or generating layout, satisfying constraints, and producing pacing jointly, with evaluation focused on robustness and generalization (e.g., across unseen dungeon sizes, objective graphs, and seed conditions) rather than only stylistic mimicry.[\[180\]](#) In such systems, the use of constraint-satisfaction-based generation methods—such as WaveFunctionCollapse variants that operate as constraint satisfaction for PCG-ML—can provide a mechanism for enforcing allowable structures while supporting coordinated generation objectives.[\[192\]](#) Together, these ideas motivate a unified optimization perspective in which layout, constraints, and pacing are treated as mutually interacting components of the same procedural synthesis problem.[\[181\]](#)[\[180\]](#)

BSP/regeneration layer as constraintsatisfaction bridge (adjacency/region graphs)

Binary space partitioning (BSP) is commonly used as a rapid “regeneration” layer for procedural game level design, because it provides a structured way to convert

spatial intent into partitioned, constraint-satisfying space.^[1] In BSP, a space (or surface) is recursively divided into two subsets over multiple iterations, producing a corresponding binary tree representation that can be used as an intermediate data structure for downstream layout logic.^[2] This makes BSP particularly useful as a bridge between high-level dungeon goals (e.g., required room placement, navigable layout constraints, and separation between functional regions) and low-level geometry needs (e.g., collision-free partitioned space).

By building a hierarchy of partitions, BSP can also support the derivation of adjacency and region-aware representations—information that is difficult to obtain from purely authored or manually constructed topology. The generated partition structure can inform adjacency graphs (which regions connect through openings or shared boundaries) and region-focused placement rules (placing gameplay elements according to the constraints and semantics of each partitioned area). As a result, BSP-generated spatial structure can feed later playability and pacing validation steps that depend on how areas connect and how players are expected to move through the level.^{[1][2]}

Research gaps in Zeldalike PCG evaluation (toward publicationready metrics)

Research on procedural content generation (PCG) in “Zeldalike” games has produced a variety of generation approaches, including evolutionary methods for dungeon maps with mission-locked doors,^[80] as well as broader studies surveying techniques for procedural generation in games.^{[81][85]} However, an important gap remains in how these systems are , particularly with respect to Zelda-like design goals (e.g., dungeon structure, exploration affordances, and puzzle variety) and the development of evaluation methods suitable for publication-ready, comparable metrics.^[85]

Several lines of work indicate the range of PCG targets that could be assessed, from dungeon maps to terrain and world elements. For example, proposals include using procedural generation to create new dungeons and puzzles for future Zelda games.^[79] Related experimental systems generate pseudo-3D terrain on tile grids using methods inspired by Zelda games.^[82] Conceptual work also imagines scaling Zelda: Breath of the Wild’s mechanics toward “infinite” maps populated by procedurally generated content such as shrines.^[83] These directions highlight that evaluation must account not only for whether generated layouts are playable, but also for whether they capture gameplay characteristics associated with open-world design and player experience.^[84]

Despite this diversity, the literature does not yet provide a widely adopted, standardized evaluation framework for Zelda-like PCG. While surveys document the current state of procedural dungeon generation research and summarize PCG algorithm categories, they primarily emphasize algorithmic approaches rather than rigorous, community-accepted metrics for judging quality across different generation systems.^{[81][85]} Similarly, studies examining open-world games (including The Legend of Zelda: Breath of the Wild) underscore the role of context and player-facing factors, suggesting that evaluation may need to incorporate measures aligned with player ex-

perience rather than purely technical properties of generated levels.[\[84\]](#) Closing this gap would likely require publication-ready metrics that can be computed consistently across PCG systems and that correlate with Zelda-like qualitative goals.

Zelda-like PCG pipeline in Unreal Engine

PCG system overview (Zeldalike goals)

Procedural content generation (PCG) in game level design refers to the algorithmic or computer-assisted creation of game content, either automatically or with designer input, instead of solely relying on manual authoring.[\[393\]\[395\]\[396\]](#) In the context of -like action role-playing games, PCG is commonly discussed as a way to produce large amounts of structured level content while maintaining the genre's emphasis on guided exploration, solvable layouts, and purposeful progression.[\[140\]\[141\]\[142\]](#)

A -like PCG system is often framed around establishing clear player progression goals and then generating content that remains controllable under those constraints. For example, lock-and-key approaches specify paths of progression alongside constraints that define what the player can access next, which can then be used to generate levels or dungeon layouts procedurally rather than leaving progression to chance.[\[401\]\[402\]](#) Related dungeon-generation work likewise emphasizes producing levels where “subsections are controlled and completable,” leveraging the fact that dungeon structures are relatively straightforward to represent in data-driven systems.[\[143\]](#) In this view, PCG targets dungeon topology and related gameplay objects, generating elements such as layout structure and gameplay-relevant entities that support the intended sequence of discovery and challenge.[\[405\]](#)

Beyond progression structure, -inspired procedural approaches also align PCG with traditional level-design principles. Retrospective research on using PCG for level generation highlights how classical design considerations—such as critical-path thinking and risk-for-reward dynamics—can influence procedural generation algorithms, helping generated spaces better approximate the player experience associated with authored dungeons.[\[398\]\[404\]](#) This goal-oriented framing supports practical production benefits as well: PCG can create more dungeons in less time, potentially reducing development costs while increasing replayability through variation in generated content.[\[142\]\[140\]](#)

Unreal Engine integration (editor vs cook time vs runtime)

In Unreal Engine–based procedural content generation (PCG) for game levels modeled on the “dungeon” structure common to , generation can be performed at three primary stages: in-editor, at cook time, or at runtime. The choice affects both correctness (e.g., producing playable layouts) and iteration speed (e.g., debugging generator logic before shipping).[\[39\]\[27\]\[41\]](#)

Editor-time generation (Construction Script / editor execution). Unreal's Construction Scripts are intended to run only in the editor; their outputs are then “baked” into cooked builds. As a result, editor-time generation is typically recommended when

the generated content can be validated and finalized ahead of play—allowing developers to quickly iterate on generator parameters and immediately inspect the resulting level structure.[\[39\]](#) This approach pairs well with graph- or structure-driven validations, since dungeon layouts can be analyzed directly from their topology using metrics such as connectivity and solvability before playtesting.[\[30\]\[29\]](#) For Zelda-like designs—often involving progression and lock-and-key constraints—editor-time generation can also support systematic checks that locked progression is consistent with key placement and routing from the entrance.[\[36\]\[33\]](#)

Cook-time validation and baking. When levels are generated prior to shipping, it is important to validate that the “editor results” still behave correctly after being cooked. Unreal workflows emphasize cook-time testing (e.g., cooking the content for a target scenario and verifying behavior) to ensure the baked generation output matches runtime expectations.[\[39\]](#) This stage can incorporate automated evaluation frameworks that measure both reliability (consistent validity) and expressivity (ability to produce varied yet constraint-compliant dungeons), using both standard metrics and domain-specific measures such as playability and robustness.[\[31\]\[29\]](#) For instance, automated dungeon-generation benchmarks and evaluation methodologies can be used to quantify whether generated layouts satisfy task constraints relevant to Zelda-like progression.[\[43\]\[31\]](#)

Runtime generation (during gameplay). Runtime generation is appropriate when the game requires on-demand content variability, dynamic replayability, or the ability to “re-roll” levels in response to player experience. While runtime generation can be designed to animate the generation process, this increases the need for real-time validation to avoid producing invalid or unfun content.[\[40\]](#) Consequently, runtime approaches are often paired with generation systems that produce constraints-compliant layouts and then perform immediate feasibility checks—such as reachability of rooms, combat pacing, and puzzle solvability—before presenting the dungeon.[\[41\]-\[27\]](#) In systems where the generator effectively creates gameplay “cycles” and then translates them into a playable dungeon, runtime generation may also require failure-mode testing to prevent dead ends, broken progression, or pacing that diverges from authored-feeling patterns.[\[34\]\[28\]](#)

Practical guidance for choosing a stage. A common strategy is to generate levels offline (in-editor and then verified at cook time) when the generator’s outputs can be thoroughly validated ahead of release using structural metrics and constraint checks, improving reliability and iteration speed.[\[39\]\[30\]\[29\]](#) Runtime generation is typically favored when diversity must be produced per session or per player, but it should include lightweight, domain-specific validation to ensure generated layouts are playable and “feel” authored—especially for progression systems resembling lock-and-key dungeon structures.[\[27\]\[36\]\[41\]](#) Across all stages, developers can use multi-objective evaluation and metric reporting (including both general and Zelda-like domain properties) to systematically measure generator quality and detect failure modes early.[\[31\]\[32\]\[29\]](#)

Evolutionary algorithms for Zeldastyle level generation

Evolutionary algorithms have been used to generate procedural “Zelda”-style dungeon layouts, including structures that support gameplay mechanics typical of The Legend of Zelda series. One approach uses the evolutionary search itself to evolve dungeon layouts that can provide missions and gameplay challenges based largely on the resulting structure, such as locked-door puzzles commonly associated with the series.[\[339\]](#) Similarly, another method presents an evolved dungeon generation system for Zelda-like games, explicitly incorporating lock-and-key puzzles as part of the evolutionary design process.[\[340\]](#)

Research in this area also evaluates evolutionary dungeon generation in a discrete 2D Zelda environment, measuring properties of the generated levels such as the number of elements, their locality, and how well structured these elements are within the dungeon layout.[\[342\]](#) Related work describes evolutionary algorithms selecting and shaping levels for skill-based games—including The Legend of Zelda—demonstrating that evolution can be applied to procedural level generation for platformers and dungeon-based designs.[\[343\]](#)

More broadly, evolutionary optimization has been combined with other generation inputs (such as those derived from Wave Function Collapse, or WFC) to improve the quality of Zelda-style dungeon layouts. In this line of research, evolutionary optimization over WFC inputs is reported to enhance generation quality in domains where higher-level properties emerge from lower-level constraints.[\[344\]](#)

Combat and encounter pacing in procedural dungeons

Combat and encounter pacing in procedural dungeons is typically designed to reinforce a play loop in which the player explores, learns the spatial layout, and then prepares for subsequent “locks” and progression gates. One approach begins by building a connectivity model of the dungeon geometry: the centers of room locations are connected using Delaunay triangulation, creating a graph in which rooms are linked to one another and can be scheduled for encounters along paths of traversal.[\[53\]](#)

To support progression, dungeon logic can couple encounter placement to explicit lock/key rules. For example, a lock may be an encounter (such as a combat with multiple enemies), while the required key is an item that enables resistance or preparedness (such as a potion of poison immunity). This makes it more likely that players obtain needed capabilities before facing the corresponding challenge, reducing the odds that they experience unavoidable difficulty spikes due to random generation.[\[54\]](#)

Encounter pacing is further influenced by how frequently additional random encounters are triggered after the player interacts with dungeon content. In one method, acquiring or discovering a “room key” and its associated lock state increases the odds of further encounters, meaning that the dungeon’s pacing can be modulated by tracking how often players open locks or disable hazards.[\[55\]](#) In research on controllability, these systems can be parameterized using both global difficulty/structure variables

(such as lock/key depth and loop count) and “micro-pacing” factors that weight safety and treasure rewards relative to player traversal distance, with the overall goal of evaluating how well the resulting pacing remains controllable.[\[56\]](#)

Spatial progression can also be enforced by restricting access between regions. If two sets of rooms are connected by only a single corridor, one side can be placed behind a lock that requires a key obtained from the other region. This ensures that players learn the layout and complete prerequisite objectives before reaching gated combat encounters that would otherwise be reachable too early.[\[58\]](#) In practice, pacing logic may be implemented by marking key nodes the party is expected to pass through (e.g., nodes representing locations critical to route planning), and then assigning encounters—such as combats or locks—relative to that traversal structure.[\[57\]](#)

Graph/feature pipeline structure for Zelda-style spatial rules

Preserving hub ’ region ’ revisit loops with deterministic seeds

Zelda dungeon grammar as hard designer-authored constraints (pacing, readability, setpieces)

Validator pipeline: solvability, traversal clarity, and flow checks

A key component of procedural generation for Zelda-style levels is a validator pipeline that enforces “Zelda dungeon grammar” as hard constraints rather than allowing free-form variation. Even when a generator produces dungeon geometry procedurally, the layout must preserve designer-authored pacing beats, critical-path readability, the hub’region’revisit loop, and iconic set-piece transitions[\[152\]\[153\]\[154\]](#) To ensure that each generated dungeon remains playable and communicates its structure to the player, the pipeline typically applies multiple checks before a layout is accepted.[\[152\]\[154\]](#)

First, a solvability check ensures that there exists at least one complete progression route through the dungeon that satisfies all gating requirements and unlock conditions, producing a consistent critical path from entry to exit.[\[152\]](#) Second, traversal clarity checks evaluate whether the player can reliably understand where the critical path lies—such as whether landmarking, visibility, and connection structure support intuitive navigation rather than requiring exhaustive exploration.[\[152\]](#) Third, flow checks confirm that the dungeon’s structure supports the intended hub-driven progression model: the player must revisit the hub frequently, and each new region traversal should integrate back into an expanding critical path complexity at the hub nucleation point.[\[154\]](#) When any stage fails, the generator can be rerun, constrained further, or adjusted to meet the designer-specified progression loop and readability requirements for that seed.[\[152\]\[154\]](#)

Feature/graph pipeline structure for Zeldastyle spatial constraints

In Zeldastyle dungeon and overworld generation, a level's spatial layout is often treated as a constrained assembly problem rather than purely random content creation. Conceptually, a pipeline can be structured around a feature/graph model in which (e.g., rooms, corridors, trial pits, or landmark sub-areas) are treated as nodes and connections, and are enforced when binding those nodes into a coherent map.

A typical approach begins by (1) defining a feature library whose elements are parameterized by size, entrance/exit ports, and adjacency rules; (2) building a connectivity graph that represents allowable paths and region-to-region traversal; and (3) running an embedding or “placement” phase that maps the abstract graph onto a concrete tile/grid or navigation mesh while respecting Zeldastyle constraints such as fixed entrances, line-of-sight or silhouette expectations, chokepoints, and world/region boundaries. Hand-crafted layout logic can be retained at the zone level (for example, bespoke overworld regions or landmark areas), while still using procedural methods to generate sub-areas like pits of trials or maze-like traversal zones, blending deterministic authored structure with algorithmic variation.[\[144\]](#)

From an implementation perspective, procedural generation is often described in general terms as using algorithms to create data, frequently modeled as pure functions that transform inputs into generated outputs.[\[145\]](#) In a feature/graph pipeline, this idea maps cleanly onto the generation stages: the constraint graph and placement parameters can serve as inputs, while deterministic solvers (or seeded stochastic methods) produce the final spatial layout. This makes it easier to enforce consistency across passes (e.g., corridor width matching between connected rooms, door alignment, and region boundary adherence) and to regenerate layouts reliably from the same high-level constraints.

Generator degrees of freedom: geometry variation within fixed grammar

In procedural Zelda-style dungeon generation, the “degrees of freedom” for level creation are often constrained by a fixed underlying rule set (or grammar), while still allowing meaningful variation in spatial layout. One approach is to use a classic 2D top-down dungeon model—drawing on roguelike-inspired principles and functioning similarly to —as the generator's baseline representation.[\[87\]](#) Within that fixed structure, geometry can vary from run to run (e.g., room and corridor shapes, placements, and connectivity) while preserving the overall gameplay intent implied by Zelda-like dungeon progression.[\[86\]](#)

These constraints are especially important for maintaining recognizable Zelda features tied to dungeon structure and access. Zelda dungeons frequently require specific in-dungeon milestones—such as a “boss key” that gates progress to subsequent areas—so a generator that varies geometry must still ensure that the resulting layout supports the intended gating and traversal flow.[\[86\]](#) Research and design commentary on “Procedural Zelda” also emphasizes that the system purposely builds on established Zelda design conventions and assets from earlier games (notably) to retain the feel of the original while enabling procedural construction.[\[90\]](#)

More broadly, procedural generation is used to increase replay value and variety, but it does so by generating new instances within boundaries set by the game's underlying design philosophy and structural rules. Studies of Zelda dungeon design, such as work analyzing design philosophies and dungeon elements like boss keys, highlight how consistent structural patterns support player understanding and progression even when the environment changes.[\[89\]](#) In this framing, geometry variation is therefore treated as controlled freedom within a fixed grammar: the generator can alter geometric details while maintaining the constraints necessary to produce Zelda-like dungeons that remain navigable and progression-consistent.[\[86\]\[87\]\[89\]\[90\]](#)

PCG pipeline stages: topology/geometry generation vs validation gates

In Unreal Engine's Procedural Content Generation Framework (PCG), level creation is commonly organized into separate pipeline stages: (1) topology/geometry generation, followed by (2) validation gates that filter or reject generated results. PCG is a graph-based toolset used to create procedural content and tools within Unreal Engine.[\[61\]](#) With UE5.2, PCG's graph-based workflow is positioned as a way to populate game worlds efficiently by driving generation through interconnected nodes and rules.[\[59\]](#)

To support reproducible generation in a "Zelda-like" design context—where layouts must respect structural progression patterns such as hub ' region ' revisit loops and lock-and-key gating—PCG graphs are typically configured so that all stochastic elements are deterministic. Seed-based generation uses a single input value to deterministically produce the same output every time, enabling consistent world generation and sharing of results.[\[62\]](#) In practice, this is often implemented by explicitly setting deterministic seeds on all PCG nodes that perform randomness: for example, selecting each node with a seed setting and switching it into a deterministic seed mode.[\[60\]](#)

After deterministic generation, validation gates are introduced as a distinct stage to ensure that the produced topology and resulting geometry remain playable and consistent with puzzle-state constraints. Rather than treating generation and correctness as the same step, the pipeline can be structured so automated checks run after placement and connectivity are established—e.g., validating navigation reachability (such as navmesh accessibility across critical routes) and enforcing puzzle-state or progression feasibility constraints (such as ensuring required conditions exist in the reachable order implied by lock-and-key progression). This separation of "generate first" (topology/geometry/materialization) and "verify second" (reachability and constraint checks) helps maintain Zelda-inspired structure while still benefiting from deterministic, graph-driven procedural generation.[\[59\]\[61\]\[62\]](#)

Adjacency graph extraction and preservation across generation steps

Adjacency graph extraction is commonly used to preserve the structural relationships produced by earlier stages of procedural dungeon generation for use in later layout

and validation steps.[\[446\]](#) In pipelines that start from a high-level topology (such as those produced by algorithmic design or automated planning), adjacency information—often represented as a graph of rooms and corridor connections—provides a compact, reusable intermediate representation that can drive region-aware placement and support downstream analysis.[\[443\]](#) After topology creation, additional procedural methods can map the graph into the game’s spatial constraints and then generate the actual room and hallway geometry required by the level.[\[444\]](#)

Binary space partitioning–based (BSP) approaches are particularly useful for this purpose because they act as a rapid, tunable space-partitioning and regeneration layer that converts design “intent” into constraint-satisfying, collision-free geometry.[\[443\]](#) This makes it feasible to preserve adjacency relationships while the dungeon is refined into a physically consistent layout, yielding data products such as adjacency graphs and other region- or partition-aware information that can be consumed by subsequent playability and pacing validation steps.[\[443\]](#) More broadly, comparative studies of multiple procedural dungeon generation algorithms—such as DFS and BSP—examine how different methods affect performance and design trade-offs within end-to-end generation pipelines, including the ability to maintain useful intermediate structure across steps.[\[446\]](#)

Solvability and player-understandable flow validation (navmesh reachability, deadlock checks, loop invariants)

To ensure that procedurally generated dungeons remain playable, designers typically apply post-generation validation focused on solvability and player-understandable navigation. In Zelda-style level layouts, this validation is used to preserve a “dungeon grammar” of designer-authored pacing and spatial logic—such as critical-path readability and a hub’region’revisit loop—while allowing the generator to vary geometry only within those hard constraints.[\[146\]\[147\]](#) Rather than treating layouts as merely aesthetic, the pipeline enforces that each generated instance (seed) yields a coherent progression where the intended sequence of discoveries and returns remains understandable to the player.[\[146\]\[147\]](#)

A common approach is to validate reachability directly on the navigable representation used by the game (e.g., a navigation mesh or equivalent traversability model). After the dungeon layout is assembled, the validator performs reachability analysis from the dungeon entry to confirm that all required lock-and-key dependencies can be traversed in the intended order, preventing softlocks (where the player can reach a locked goal but cannot obtain the required item/key) and deadlocks (where the player becomes unable to progress because required access never becomes available).[\[146\]\[147\]](#) This reachability checking is also used to verify the critical path: even when a layout contains optional spaces or loops, the required route should be navigable without requiring ambiguous backtracking, so that the hub’region’revisit structure remains legible across seeds.[\[146\]\[147\]](#)

To control the effects of adding cycles (shortcuts, loops, or return paths), some pipelines start from an acyclic “perfect maze” baseline (a spanning-tree maze) and then insert loops only when needed. In this model, the system first generates an

acyclic baseline and then validates solvability and solution-characteristics under the same navigation cost model before adding cycles via controlled modifications.-[\[511\]](#)[\[512\]](#) The loop insertion step is constrained by invariants—such as preserving dependency order for lock-and-key progression, maintaining target properties of branching/dead-end frequency, and ensuring that added shortcuts do not invalidate the designed progression structure.[\[511\]](#) Conceptually, this prevents the dungeon from degrading into a layout where movement is possible but progress is not (for example, creating corridors that effectively loop without advancing the player's objective), which would undermine the intended hub-and-spoke pacing.[\[149\]](#)

Puzzleroom type generation (algorithms/ML options integrated with topo + pathfinding)

Procedural generation of puzzle rooms in games such as –style dungeons can be approached by combining dungeon topology and pathfinding constraints with content-generation algorithms that explicitly encode puzzle logic. One approach is to generate dungeon “level requests” (i.e., structured intents for regions and game-play beats) and then derive rooms, encounters, and enemies from those requests, allowing puzzle rooms to be treated as first-class room types within the generator's pipeline.[\[418\]](#) Related designs emphasize constructing grid-based modular dungeon layouts while defining room setup, exits, and the procedural logic that places connectivity and gameplay features consistently.[\[420\]](#)

Puzzle-room generation can be integrated with topology and navigation by using algorithms that ensure both reachability and rule-based gating. For example, evolutionary computation has been proposed to procedurally generate dungeon maps featuring locked-door missions, which naturally aligns with puzzle mechanics where access is conditioned on solving objectives.[\[421\]](#) More broadly, baseline randomized dungeon layouts built from rooms, corridors, and walls can serve as the topological substrate; puzzle rooms can then be inserted or parameterized as special rooms within that structure while preserving overall dungeon connectivity for traversal and backtracking.[\[422\]](#)

Beyond structural placement, puzzle rooms may be generated as rooms containing specific mechanical or logical patterns rather than generic hazards. Such patterns can include magic-themed puzzles, locked vault spaces, and mechanical traps, enabling richer puzzle variety than simplistic “damage tile” gameplay.[\[423\]](#) To support these designs, puzzle-generation logic can be coupled to pathfinding by marking traversal states (e.g., keys obtained, switches triggered) and ensuring puzzle dependencies produce valid routes through the dungeon's graph.

In addition to classical algorithmic methods, procedural puzzle generation can leverage machine learning to choose puzzle-room layouts or validate them against navigation and constraint satisfaction criteria, using the dungeon topology (graph/room adjacency) and pathfinding outcomes as part of the training or inference loop. Practical ML options include generating candidates (puzzle configurations, trap placements, lock/key placements) and filtering or ranking them using reachability tests (via pathfinding) and constraint checks (e.g., ensuring there exists at least one solution

path given the puzzle's state transitions). This kind of “generate-and-test” integration parallels the use of structured PCG logic for dungeon construction while extending it toward puzzle-specific correctness and solution validity.[\[418\]](#)[\[420\]](#)[\[421\]](#)[\[422\]](#)

Topologyaware ML objectives to preserve graph characteristics (persistent homology/persistence images, loop persistence regularization)

Topologyaware training objectives for graph structure preservation

TopologyPreserving Constraints in GraphBased Level Generation (citation: 1, 2)

Topology-preserving approaches to graph-based level generation aim to ensure that the generated content retains salient topological structures present in a target domain. In this setting, topological features are extracted from graphs using techniques from topological data analysis, such as persistent homology, which summarizes how connected components and higher-dimensional cycles appear and disappear across a range of scales. These features can then be incorporated into learning-based generators to control or regularize the produced graphs.[\[359\]](#)[\[360\]](#)

One practical strategy is to generate and combine topological signatures computed from multiple filter functions, learning how these summaries should contribute to the final generation objective.[\[359\]](#) Related work also emphasizes that topological computation can be used in the learning loop to maintain the essential structure of graphs under transformation or generation.[\[358\]](#)

In topology-aware generation, explicit constraints can be imposed using persistent homology. For example, methods that incorporate topological features via persistent homology may sparsify or restructure an intermediate representation while “revealing” essential topological structures, indicating that the generation process can be guided toward preserving those properties rather than merely optimizing for conventional graph statistics.[\[361\]](#)

More generally, controlled graph generation frameworks have been proposed to provide fine-grained control over topological properties of generated graphs, including the ability to target specific persistent-homology-related characteristics.[\[362\]](#) In addition, differentiable pipelines can connect learning objectives directly to topological summaries: by enabling backpropagation from a loss defined on the persistence diagram, such systems treat the birth and death of topological features as learnable targets, supporting end-to-end optimization toward topology preservation.[\[363\]](#)

TopologyAware Training Losses for Graph Structural Fidelity (citation: 1, 2)

In procedural content generation (PCG), machine-learning-based level generators are often trained to produce playable maps or layouts that are representative of a target design style. However, when generation is extended beyond the distribution of the training data, there is a risk that important structural characteristics—such as the topology or connectivity patterns of the underlying representation—may not be

faithfully preserved. This concern has motivated research directions that emphasize in PCG settings, including the design of training objectives intended to retain the structural properties of the level graph during generation.[\[364\]](#)[\[365\]](#)[\[366\]](#)

Topology-aware training losses are a class of approaches proposed to address this mismatch by incorporating information about the graph's structure directly into the optimization process. Rather than relying only on pointwise reconstruction or generic adversarial objectives, such losses are intended to penalize outputs that deviate from the input or reference graph's higher-level topology (e.g., preserving connectivity patterns that correspond to navigability, spatial relationships, or other graph-based design constraints). In the context of PCG—where content is produced automatically via algorithms and increasingly via deep learning methods—these losses are framed as mechanisms for aligning the generator's learning objective with the structural aspects that define quality and consistency of the generated game content.[\[365\]](#)[\[366\]](#)[\[367\]](#)

More broadly, surveys of deep learning methods for game content generation describe how neural approaches can generate content directly or indirectly, and they highlight that maintaining faithful structure is a recurring challenge when content is represented as graphs or other structured data. Within that landscape, topology-aware objectives are relevant for ensuring that generative models remain consistent with the intended underlying representation of the level, supporting better preservation of graph-derived properties even when generation is performed by learned models rather than by hand-designed procedural rules.[\[364\]](#)[\[365\]](#)[\[366\]](#)

Persistent homology / persistence image loss over traversal cost filter functions

Topology-aware approaches to procedural content generation (PCG) incorporate tools from computational topology—particularly persistent homology—to align generated game levels with structural properties of a target level design. In this setting, losses are formulated to compare the topology of candidate and reference level “graphs” or spatial representations by computing persistent homology under a family of filter functions. One proposed strategy uses (vectorized summaries of persistence diagrams) computed over filter functions related to traversal cost, so that the optimization penalizes mismatches in connectivity and cycle “loop” persistence rather than relying only on geometric plausibility.[\[368\]](#)[\[370\]](#)

To support differentiable training, persistent homology can be computed using a real-valued, learnable filter function, enabling the filter to adapt to aspects of level structure relevant for play. Under such a framework, the training objective directly encourages preservation of topological features that correspond to desirable traversal and exploration patterns (e.g., maintaining appropriate connected components and loop structures).[\[371\]](#) Persistent homology then acts as the central “workhorse” for guiding the hybrid generative/procedural pipeline toward layouts that retain the intended underlying graph-like organization.[\[370\]](#)

Finally, these topology-driven losses are typically combined with explicit constraints or regularizers designed to preserve playability invariants under perturbations. For

example, objectives can be coupled with terms that maintain reachability properties among key regions and prevent undesirable game states such as deadlocks, ensuring that topology-aware similarity translates into robust gameplay-relevant structure.-
[\[368\]](#)[\[370\]](#)

Pathfinding robustness in PCG dungeons (walkable paths, links between regions)

In procedural content generation (PCG) of dungeon levels, ensuring that generated layouts support reliable navigation is often treated as a problem—both for runtime agent movement and for the generation process itself. Pathfinding can be defined generally as the task of finding an uninterrupted, traversable route between two or more points, preferably an optimal one.[\[263\]](#) Because PCG may produce many candidate maps, designers and researchers often aim for methods that not only find routes, but also guarantee that connectivity and walkability constraints are satisfied.

One approach is to represent the dungeon as a connected graph (for example, rooms as nodes and passages as edges) and then convert or “materialize” that abstract connectivity into walkable space. Graph-to-corridor conversion can be performed by constructing geometric connections for each edge, such as straight segments or L-shaped turns between the appropriate room boundaries.[\[253\]](#) Related methods describe turning a connected graph of arbitrary shape into a grid-based layout suitable for further processing, reflecting the common practice of combining graph structure with grid-based navigability.[\[255\]](#)

PCG workflows may also incorporate pathfinding ideas directly during generation. This is reflected in the view that pathfinding is not only used a level is built for AI traversal, but can also be used as part of the paths themselves, thereby enforcing navigability requirements during generation.[\[259\]](#) Practically, this can involve building pathable corridors/paths that link regions so that, as the dungeon is instantiated, there are valid traversal routes between connected areas rather than disconnected or blocked regions.[\[252\]](#)

To manage both correctness and performance in larger maps, hierarchical and graph-structured navigation schemes are frequently used. For example, Hierarchical Pathfinding A* (HPA*) divides a map into regions and searches at multiple abstraction levels, improving performance while still enabling path queries over the underlying connectivity structure.[\[260\]](#) In the same general spirit, the navigation mesh (NavMesh) represents accessible areas as a graph, where nodes correspond to walkable regions and edges correspond to valid transitions, providing a structured connectivity model that can be generated and queried for navigation.[\[261\]](#)

When the dungeon is represented as a grid, robust path availability may be improved by constraining corridor geometry and smoothing routes to reduce unnatural or collision-prone traversal paths. Techniques have been proposed that compute smooth path guidance fields (e.g., via eigenfunctions or Laplacian-based smoothing), then bias graph construction so resulting paths form curves and maintain navigability constraints.[\[258\]](#) More broadly, graph-based dungeon generation can be evaluated

or verified using heuristics designed to check whether procedurally generated graph dungeons meet desired traversability properties.[\[256\]](#)

Validator timing and stages

Deterministic PCG data model for Zelda constraints (locks/keys, tags/affordances, region budgets)

In deterministic procedural generation of Zelda-like game levels, the level is modeled as a graph whose nodes represent room centers (or other key locations) and whose edges represent guaranteed spatial connectivity. A Delaunay triangulation over room center points can be used to produce an initial fully connected adjacency structure, which can then be constrained or specialized to enforce Zelda-style navigation patterns such as “hub ’ region ’ revisit” loops while remaining reproducible under fixed random seeds.[\[66\]](#) In this approach, the graph is not only a geometric scaffold; it also serves as the primary data model for quest and spatial rules, supporting a mapping step that places the abstract graph into the intended playable game space and a separate step that generates geometry for rooms and hallways.[\[67\]](#)

Zelda-style lock-and-key progression can be encoded directly into the graph by annotating specific nodes and/or edges with lock and key semantics. For example, a lock can be defined as an encounter or interaction gate (e.g., a combat encounter with 1–2 giant poisonous rats) and resolved by a corresponding key item (e.g., a potion granting poison immunity), establishing an explicit rule linking item acquisition to traversal or access.[\[68\]](#) At the algorithmic level, nodes that correspond to “key locations the party will pass through” are typically marked first, enabling later placement decisions to reference a deterministic sequence of required interactions rather than emergent, unconstrained routing.[\[69\]](#) Locks can also be represented structurally through constrained connectivity—such as partition cells connected by only a single corridor—where the corridor or adjacent regions may be treated as locked away until the relevant key is obtained.[\[70\]](#)

Beyond lock-and-key semantics, deterministic Zelda-like generation commonly includes additional tag/affordance metadata and region-level constraints to control how traversal, content placement, and revisitation unfold across the level. These constraints can be implemented as region budgets and other graph-derived requirements, ensuring that the second stage of room-and-hallway geometry generation remains consistent with the abstract traversal and gating rules established earlier in the pipeline.[\[67\]](#)

BSP collisionfree partition stage (constraintdriven room/corridor placement, iteration sandbox)

Binary space partitioning (BSP) is commonly used in procedural content generation to support dungeon and level construction by recursively dividing a space into two subsets, yielding a collision-free partitioned representation of the playable area.-[\[438\]](#)[\[436\]](#) In a typical dungeon pipeline, the process begins with spatial division

using BSP, after which rooms are placed in an ordered fashion and further refined using rule-based and stochastic methods.[\[440\]](#) Because BSP partitions the map into non-overlapping regions, each region can be treated as a candidate “leaf” area for subsequent constraint-driven placement of rooms and corridors, helping ensure that layout elements do not overlap while maintaining controllable spatial boundaries.[\[442\]](#)

In practice, BSP can be represented as a binary tree (a BSP tree), where different splitting strategies determine how the space is subdivided into smaller regions.[\[441\]](#) For example, one approach starts from a rectangular dungeon area initialized with wall cells, then builds the BSP structure to guide how the interior is subdivided for later carving or placement steps.[\[437\]](#) This kind of top-down partitioning makes it easier to enforce minimum region sizes and other structural constraints during room/corridor placement, since each partition leaf corresponds to a guaranteed minimum-sized region available for layout.[\[442\]](#)

Beyond producing usable partitions, BSP also serves as an efficient iteration sandbox for pipeline stages. Once BSP-derived partitions exist, downstream systems can quickly test and validate different constraint sets for room and corridor placement, including connectivity and pacing checks, before committing to the final graph or pacing coherence of the level.[\[435\]](#) As a result, BSP enables rapid parameterized iteration: dungeon topology can be authored or sketched with higher-level intent, while BSP continues to support subsequent stages by providing fast, collision-free spatial structure for constrained placement and verification workflows.[\[435\]](#)

“PlayercanreachwhatIthink” solvability validation (Astyle checks using same nav/cost model)

Astyle reachability constraints parity (locks, oneway/forbidden, widths)

For graph-based procedural content generation (PCG) of action-adventure game levels, reachability validation is often used to ensure that players can access the intended objectives within the generated topology. In this approach, generation is modeled on a dimensional expanded graph (DEG), and then validated using A*-style reachability checks over the same navigation/cost model used during placement of level features.[\[1\]\[2\]](#) The reachability model incorporates gameplay-relevant movement and traversal constraints such as corridor dimensions (e.g., corridor width), directional constraints (one-way passages), and forbidden or disallowed edges, as well as gating systems such as locks/keys and traversal requirements that can restrict which regions are accessible.[\[1\]](#)

Beyond basic “is there a path” testing, validation is commonly strengthened by analyzing path cost and structural properties between landmarks (e.g., thresholding landmark-to-landmark path cost and corridor-count measures). This is complemented by a connectivity “failure-mode” audit aimed at detecting issues that may not be revealed by existence checks alone, including unreachable keys or goals, topological disconnects, bottlenecks or overlapping constraints that block traversal, and other no-route failure cases.[\[1\]\[2\]](#) Techniques from graph theory such as transi-

tive closure can also be used conceptually to reason about reachability over nodes, though A*-style evaluation provides a cost- and constraint-aware version aligned with the navigation model.[\[3\]](#)

Hybrid methods that combine large language models (LLMs) with formal graph-analytic repair further highlight the practical importance of constraint-aware reachability: while LLM-driven PCG can approach parity for smaller training scales, reachability problems may persist as level size increases, motivating automated validation and repair pipelines grounded in the underlying graph model and constraints.[\[4\]](#)[\[5\]](#)

Landmark-to-landmark thresholding (path cost + corridor count)

Landmark-to-landmark thresholding is a reachability validation technique used in graph-based procedural generation to ensure that gameplay-relevant traversal is both possible and appropriately structured. In this approach, the generator first designates landmarks (e.g., key locations, room entry points, or navigation waypoints) and then evaluates traversal between them using a pathfinding method. The validation is typically performed with A*-style reachability checks that operate on the same navigation/cost model used during placement, including traversal constraints and graph edge restrictions.[\[465\]](#)[\[466\]](#)

To assess not only whether routes exist but whether they satisfy intended pacing and topology, systems often combine (1) path-cost thresholds—such as total estimated travel cost or distance—and (2) corridor-count thresholds—such as the number of corridor segments or region transitions encountered along the route. By enforcing bounds on both metrics, the generator can reject layouts where landmark-to-landmark travel is too short (indicating overly direct shortcuts) or too long/complex (indicating excessive backtracking or circuitous navigation). This is commonly paired with connectivity “failure-mode” audits, but the thresholding step specifically provides a controllable way to compare alternative placements and prevent structurally undesirable routes, including cases where connectivity may appear present while still producing problematic traversal characteristics.[\[464\]](#)[\[465\]](#)

Research on PCG workflows for 2D video game levels likewise emphasizes validating that the player can reach key elements (e.g., key and door) using A* algorithms, reflecting the broader use of path-cost evaluation to confirm that critical landmark-to-landmark interactions are feasible under the intended navigation model.[\[466\]](#) Related work also supports using simple player simulation/A* checks to verify that the next stage of the level can be reached during generation, reinforcing the idea that landmark-based traversal measurements are useful for constraining the produced level graph to behaviors compatible with the intended player progression.[\[467\]](#)

Representation mismatch safeguards in pathfinding (tile state normalization, start/goal passability, coordinate/state pruning)

In Zelda-like procedural level generation, a common failure mode in automated playability “solvability” checks is a representation mismatch between the generated dungeon and the pathfinding model. Even when a logically valid route exists, dis-

crepancies such as undefined or edge tile types, inconsistent walkability masks, or misclassified start/goal passability can cause the pathfinding pass to return false “no path” results.[\[274\]](#)[\[276\]](#) To reduce these issues, PCG pipelines are often augmented with safeguards that normalize the generated layout into the exact tile-state conventions expected by the solver, including consistent walkability semantics, unified coordinate indexing, and movement/neighbor rules.[\[282\]](#)

A typical approach is to enforce tile-state normalization (e.g., mapping every tile to a well-defined state in the solver’s schema) and to guard boundary conditions through clamping or padding so that tile lookups and neighbor enumeration remain valid at the grid edges.[\[274\]](#) For the pathfinding step itself, layouts may be converted into graph representations—such as using rooms as nodes and corridors as edges—requiring careful alignment between the PCG’s structural assumptions and the graph construction logic used by the solver.[\[276\]](#)

Start and goal validation is also critical. Before running solvability ranking, the pipeline can explicitly verify that the start and goal locations satisfy the solver’s passability criteria (i.e., are not placed on tiles the solver treats as blocked), and then re-run validation after correcting any inconsistencies.[\[274\]](#)[\[282\]](#) In addition, to avoid failures caused by overly strict or incomplete state pruning, fallback validators can re-run pathfinding with corrected start/goal positions and relaxed constraints before rejecting a candidate dungeon.[\[274\]](#) These measures help ensure that solvability evaluation reflects the actual navigability of the generated dungeon rather than artifacts of representation mismatch, coordinate/state pruning, or boundary handling errors.[\[274\]](#)[\[282\]](#)

Adversarial and solverconsistency validation (boundary clamping, multiheuristics, bidirectional A agreement, postroute simulation)

Tilestate normalization & grid convention alignment

Tilestate normalization & grid convention alignment is a crucial step when using pathfinding systems in procedural game level generation, particularly to avoid erroneous “no path” failures that can arise from mismatches between a generator’s tile-state representation and the pathfinder’s expectations.[\[277\]](#)[\[278\]](#) A common approach is to ensure that the procedural output is normalized to the pathfinder’s exact grid/tile/state conventions, including consistent walkability masks, correct start/goal solidity checks, unified coordinate indexing, and matching movement-cost/neighbor rules.[\[277\]](#) Without this alignment, a layout that is logically traversable by design may be evaluated as disconnected by the solver due to representational differences.[\[277\]](#)

When constructing or importing a navigation representation, developers may create a navigation mesh either by baking it through an engine-specific server or by manually providing vertex data and convex polygon indices.[\[277\]](#) Regardless of method, the same normalization principles apply: tile states must correspond exactly to the solver’s interpretation of walkable versus blocked regions, and coordinate mappings must be identical across the generator, navigation grid, and agent movement logic.-[\[277\]](#)

Because pathfinding outcomes can also depend on how tile costs and weights are interpreted, procedural systems often introduce controlled randomness into tile weights to produce more organic routes, including around interactable obstacles.[\[279\]](#) However, cost randomization must still respect the pathfinder's convention alignment; otherwise, the added variation can compound normalization errors and produce solver-versus-simulation disagreement, including false “no path” results.[\[279\]](#)[\[278\]](#) To verify correctness, validation suites are typically used to test edge cases such as start/goal boundary conditions and scenarios that should remain solvable when keys or other state-dependent mechanics are required.[\[277\]](#) Complementary consistency checks—such as comparing forward and bidirectional A* results and simulating the produced route afterward—help ensure that the generated path is truly executable under the same rules used during pathfinding.[\[277\]](#)[\[278\]](#)

Start/goal passability & edgecase gating

In procedurally generated Zelda-like levels, ensuring reliable start-to-goal passability is typically treated as a primary constraint: the generator should produce paths and traversal logic that guarantee the player can reach key locations while still enforcing the dungeon's intended progression. Approaches to procedural level generation for roguelike/dungeon-style experiences often formalize the level as a combination of topology, geometry, and gameplay-related objects, meaning that passability is not only a geometric concern but also depends on how gameplay elements (e.g., obstacles, gates, and interaction-relevant objects) are placed and connected.[\[117\]](#)[\[118\]](#)

Because procedural generation systems can yield unusual or unintended layouts, edgecase gating—handling rare configurations that would otherwise block progress—is usually required. For example, even if the generator produces a plausible overall structure, disconnected regions, unreachable rooms, or improperly linked traversal chains can occur; robust systems therefore incorporate mechanisms that keep generated levels diverse while also being constraint-compliant (e.g., ensuring that required gameplay transitions remain possible).[\[111\]](#)

A major practical challenge is that progress-gating quality is often difficult to evaluate objectively. Evaluating procedurally generated game levels is frequently constrained by the lack of standardized, game-independent metrics, which makes it harder to systematically detect and measure whether edgecase layouts (such as near-blocking chokepoints or accidental softlocks) have been eliminated across many seeds.[\[106-\]](#)[\[112\]](#) As a result, many systems rely on search, validation, and constraint-checking steps—often motivated by established level-design principles (such as critical paths and risk/reward structure) that can be translated into algorithmic checks for reachability and progression gating.[\[114\]](#)

Beyond traditional rule-based generation, some implementations incorporate additional sources of generation (e.g., non-LLM-based AI assistance for parts of the pipeline), but gating still depends on verifying that generated results meet traversal constraints. In such systems, AI may replace or support portions of the procedural generation process rather than directly guaranteeing playability, so explicit passability validation remains necessary.[\[110\]](#)[\[116\]](#)

Finally, the need for start/goal correctness interacts with the level structure model used. Methods that are geared toward Zelda-like dungeon layouts can be adapted to generate more general world structures and branching dialogue, but progression constraints (and therefore start/goal passability and gating) must still be enforced consistently in the produced structure.[\[108\]](#) Techniques spanning multiple procedural generation methods (e.g., random walks and cellular automata) can contribute to building candidate layouts, yet they still require additional evaluation or validation to prevent edgecase failures where the player cannot meaningfully progress from the starting area to the objective.[\[109\]\[118\]](#)

Fallback/repair validation before rejecting candidates

Before a procedurally generated level layout is rejected as “unsolvable,” PCG systems can perform fallback and repair validation to reduce false negatives caused by mismatches between the layout representation and the pathfinding model. Because it is not possible to know in advance whether a valid route exists without actually running a path-finding algorithm, candidate layouts should be assessed using solvability checks, but those checks should include safeguards to address common causes of “no path” results despite an apparently valid route.[\[1\]\[273\]](#)

One practical approach is to normalize tile and passability state consistently between the generator and the navigation graph, ensuring that edge cases such as undefined tiles and start/goal positions are classified with the same assumptions as the pathfinding grid.[\[1\]](#) Additionally, boundary padding and clamping can prevent out-of-range coordinates from producing accidental disconnects in the computed graph.[\[1\]](#) Graph construction can be made more robust by using multiple heuristics or validation passes so that an implementation detail does not incorrectly prune viable state transitions.[\[1\]](#)

If the initial solvability test fails, a fallback validator can re-run pathfinding after applying targeted repairs—such as correcting start/goal positions, rebuilding or adjusting the navigation graph, or relaxing overly strict constraints—before concluding that the candidate must be discarded.[\[1\]](#) When layouts rely on predefined connections between regions, static route data can also be incorporated into the routing model, and pathfinding between zones can be validated against those route tables rather than relying solely on inferred adjacency.[\[272\]](#) This “repair then re-validate” strategy helps ensure that the PCG system rejects only genuinely unsolvable dungeon candidates rather than those that fail due to representational or implementation discrepancies.[\[1\]](#)

Graphdrift and progression integrity checks (room–corridor connectivity, locks/keys feasibility, chokepoints, biconnected regions, loops/revisit)

Graphdrift refers to changes in a procedural dungeon’s underlying room–corridor structure that can alter intended progression, routing, or solvability. To mitigate this, automated integrity checks typically treat the dungeon layout as a graph—where rooms correspond to nodes and corridors to edges—and validate topological properties before downstream geometry or artistic variation is produced.[\[375\]\[379\]](#) This

approach is commonly paired with invariant-based playability simulation to confirm that the generated structure remains consistent with its intended progression logic.-
[\[375\]](#)

Room accessibility is usually tested by running a breadth-first search (BFS) across the room grid to verify that all rooms remain reachable from the dungeon's entry point.[\[377\]](#) Progression constraints imposed by doors, keys, and one-way "valves" (to force movement along a known route) can then be checked by deriving or confirming an ordering of keys and doors such that every door's key appears earlier in the traversal order; under this constraint, keys may be placed according to the required sequence.[\[376\]](#)[\[378\]](#) In practice, dungeon generation pipelines can model room adjacency using graph construction steps such as Delaunay triangulation over room centers, producing a connectivity graph that supports subsequent routing and integrity verification.[\[381\]](#)

Chokepoint analysis and related structural robustness checks can be performed by examining how vulnerable the traversal graph is to removing or bypassing specific corridors, as well as how the dungeon decomposes into strongly constrained regions. One way to support such checks is to identify room precedence relationships via a tree-based graph method, which can indicate which rooms come before others and therefore where key placement is permitted relative to locked progression.[\[382\]](#) Finally, loop and revisit behavior are validated by ensuring that repeated traversal opportunities do not accidentally enable unintended paths that bypass locks/keys or break the expected one-way constraints; routing feasibility for agents within the resulting graph can be evaluated using standard pathfinding such as A* over the room-corridor representation.[\[380\]](#)

Requiredelement + navigation twocheckpoint enforcement (dependency graph reachability ' routereadability/learnability metrics)

In lock-and-key procedural level generation, required progression typically depends on both non-spatial dependencies (such as keys, codes, and activations) and spatial traversal (being able to physically reach interactions and return to locked areas).[\[243\]](#)[\[244\]](#) A common approach enforces "required element + navigation" across two validation checkpoints: first verifying progression reachability in a dependency structure, and then validating spatial navigability using route-readability metrics.[\[241\]](#)[\[242\]](#)

The first checkpoint treats the level's progression as a dependency graph in which keys are acquired at specific nodes and locks gate traversal along locked edges; the generation process is validated by ensuring that the intended activation state reachability holds throughout the graph.[\[246\]](#)[\[241\]](#) Playability therefore requires that all keys intended to unlock later content are reachable before the corresponding locks are encountered, effectively constraining the generation to produce keys and their associated locks as consistent pairs.[\[245\]](#)[\[242\]](#)

The second checkpoint focuses on traversal quality: beyond existing topological connections, the layout must support consistent learning and navigation. After dependency reachability is satisfied, the algorithm validates spatial navigability using route-readability metrics such as walkability/topology, obstacle density, and the pres-

ence of multiple signposted routes, so required interactions are not only placeable but also consistently traversable across generated seeds.[\[241\]](#) This can be complemented by general connectivity checks—e.g., confirming that all free cells are reachable via a depth-first search—ensuring the dungeon’s navigable space is not fragmented.[\[247\]](#)

Zelda-aligned scope & pacing validators (overworld’dungeon’revisit loop coherence, global progression beats)

In Zelda-like procedural content generation, maintaining coherent and across the full gameplay loop is typically treated as a design objective rather than an emergent property. One approach emphasizes generating not only self-contained dungeons but also the transitions that link the overworld to dungeon objectives and later revisits, aligning end-to-end progression (“overworld’dungeon’revisit”) with a validator-enforced sequence of gameplay beats across the world.[\[163\]](#) Techniques developed for Zelda-like dungeon construction can also be repurposed to support broader content goals such as generating custom world structure and branching dialogue, which helps extend coherence beyond isolated rooms or floors.[\[164\]](#)

Lock-and-key procedural content generation provides a framework for structuring these globally meaningful loops. In this view, lock-and-key constraints are used to place missions and gating elements in ways that encourage intended traversal and revisit behaviors, rather than relying purely on local solvability metrics.[\[165\]](#) Related research on dungeon generation explicitly targets dungeon content while cataloging commonly used methods, reflecting the need to coordinate dungeon-scale structure with higher-level traversal and pacing expectations.[\[166\]](#)

Specific mechanisms have been proposed to enforce the feel of an “arc of progress and pacing” through dungeon generation. For example, Dormans describes a “cyclic dungeon generator” as providing a meaningful progression curve, rather than producing disconnected dungeon layouts.[\[168\]](#) More concretely, evolutionary algorithms have been introduced to procedurally generate dungeon maps containing locked door missions, indicating how algorithmic search and constraint satisfaction can be used to realize progression- and pacing-relevant dungeon structure.[\[167\]](#) However, discussions of Zelda-inspired PCG also caution against design traps that combine Zelda and rogue-like randomness without preserving the structured progression and non-random exploration rhythms associated with Zelda’s pacing.[\[170\]\[171\]](#)

Within this framing, additional attention is paid to preventing incoherent “looping” or infinite wandering that does not reproduce Zelda-like traversal intent. Conceptually, examples such as indefinitely repeating leftward movement illustrate the risk of generating overworld regions that loop without meaningful progression, thereby breaking loop coherence between areas and revisits.[\[172\]](#) Overall, Zelda’s influence on procedurally generated world behavior is often characterized as effectively bridging world design practices with procedural generation and gameplay pacing, aiming for content that follows the game’s structured beat progression across the entire play experience.[\[173\]](#)

Zeldastyle level generation workflow (PCG)

PCG navigation and layout design

Procedural content generation (PCG) for game levels often uses a bottom-up approach to build layouts from the smallest components upward, allowing the system to assemble playable spaces without manually authored room-by-room design.[\[289-1\[293\]](#) In dungeon-based games inspired by , a common application is the automated creation of multi-room areas—such as randomly generated dungeons where players descend through a series of levels and ultimately retrieve an objective item (for example, an amulet of Yendor in roguelite-style implementations).[\[290\]](#)

After rooms or other structural elements are generated, navigation constraints are typically enforced. For instance, once generated rooms are connected, pathfinding algorithms can be used to determine regions that are “safe” for placing obstacles, ensuring that the player can still traverse the layout as intended.[\[292\]](#) More broadly, PCG methods can be extended beyond simple dungeon geometry to support navigation-relevant or world-building elements, such as custom worlds and branching dialog systems.[\[291\]](#)

Beyond connectivity and routing constraints, algorithmic approaches may also search the space of possible dungeon layouts. For example, evolutionary algorithms have been used to evolve tree-structured representations of dungeons, aiming to produce levels that closely match an input configuration provided by a game.[\[294\]](#)

spritelike dungeon/tile layout generation

Spritelike dungeon and tile layouts in action-adventure roguelites are often generated using procedural dungeon algorithms that build level geometry from simple primitives such as grid-based room tiles and connected room “nodes.” One approach is to create a set of rooms on a square grid and then connect them to form a playable dungeon layout, producing results that resemble hand-designed, tile-based levels rather than fully freeform terrain.[\[326\]](#) Such systems typically support multiple levels of dungeon generation, enabling continuous replay by generating a new dungeon structure each run.[\[318\]](#)

Beyond basic room-and-corridor construction, more structured generation methods can control gameplay flow by composing dungeon elements into recognizable patterns. For example, cycle-composition techniques generate dungeons by assembling circular loops of linked rooms, using these repeating substructures to shape player movement and pacing.[\[321\]](#) Other methods use evolutionary computation—such as genetic algorithms—to evolve dungeon room configurations according to higher-level design objectives, including gameplay pattern constraints captured in an objective function.[\[322\]](#) Related evolutionary approaches have been used to generate dungeon maps that include mission logic such as locked-door objectives.[\[323\]](#)

Procedural tile layout generation can also extend to larger-scale world maps. Developers sometimes generate an overworld map procedurally while still manually specifying or constraining individual point-of-interest structure, blending algorithmic

placement with designed content where needed.[\[320\]](#) Lock-and-key frameworks are another common design lens for procedural content, where generated layouts incorporate keys and corresponding locked doors to produce consistent mission logic within the dungeon graph.[\[319\]](#)

Mission/quest graph ' room generation pipeline

A common workflow in procedural content generation for game levels is to generate a mission or quest graph first and then transform that structure into a corresponding set of rooms and spatial layout. In this approach, designers typically construct a mission graph that encodes objectives and their relationships, and then use the graph as input to place rooms onto a map in a way that reflects the intended progression.[\[335\]](#)[\[336\]](#) Rather than treating rooms as independent assets, the pipeline interprets nodes and connections in the mission graph to guide where rooms should appear and how they should relate spatially, effectively linking narrative or gameplay structure to environmental structure.[\[335\]](#)

One alternative discussed in related implementations is to represent the intermediate result as a hierarchical tree (e.g., a room hierarchy) and then visualize or layout rooms from that hierarchy. However, this is often viewed as less direct for the purposes of level generation efficiency, motivating searches for more streamlined methods that better connect high-level graph structure to concrete room placement.[\[337\]](#)

Practical considerations can also affect how the mission graph is mapped to rooms. For example, when generating environments that resemble dungeons, the corridor-based connectivity typical of dungeon layouts may not match the hall-based structure expected in residential spaces, meaning that the room-generation rules must be adapted to the target environment type even if the same mission-to-space pipeline is used.[\[338\]](#)

Dungeon graph connectivity & room placement rules

Dungeon level procedural generation that targets and commonly begins with the creation of a , which encodes how player objectives and rooms relate in sequence. In one approach, a mission graph is first generated (e.g., via graph growth or pattern-based techniques) and then interpreted to place rooms onto a map according to the graph's connections.[\[327\]](#) Related work uses graph-generation strategies such as tree-based graph construction to keep track of room precedence, which can then inform placement rules—for example, determining where a key should be located relative to rooms that must be reached earlier.[\[328\]](#) Other formalisms treat generation as graph transformation (analogous to textual substitution), where specific substructures are matched and replaced using pattern and rewrite rules to iteratively shape connectivity.[\[329\]](#)

Once a dungeon or mission graph exists, room placement typically follows from the graph's topology. A straightforward implementation places rooms on a 2D grid using the generated graph as the adjacency structure, selecting a starting room and ensuring each subsequent room is positioned and connected consistently with the underlying graph.[\[330\]](#) More formal combinations of generation methods exist as

well: missions may be produced through a graph grammar that represents sequential tasks, after which a shape grammar uses the resulting mission graph to create spatial layouts.[\[331\]](#) Studies focusing on dungeon-level mission graphs likewise propose methods that differ from earlier graph-rewriting techniques, emphasizing how mission structures can be generated to support level design constraints.[\[333\]](#)

In practice, connectivity constraints are often reinforced by explicit placement rules for traversal-critical elements such as doors and paths. For example, designers may ensure that edges in the room graph correspond to traversable connections (doors) between rooms and the main route of the dungeon, with the spatial placement reflecting which rooms attach to the primary path and where transitions occur.[\[334\]](#) These connectivity rules are often paired with puzzle pacing principles—such as using multiple rooms and sections that gradually expand into more complex, maze-like dungeon interactions—common in Zelda-style dungeon design, where disconnected segments and iterative mechanical ideas encourage exploration and progression.[\[489\]](#)[\[490\]](#)

Pathfinding constraints & topological characteristics

In procedural dungeon generation, especially for Zelda-like designs, the dungeon's topology is often represented as a graph to support reliable navigation and gameplay constraints. In such representations, rooms are treated as nodes and corridors as edges, allowing pathfinding and reachability properties to be reasoned about algorithmically.[\[450\]](#) A common approach is to generate corridors by connecting candidate room positions using a graph-search method such as A*, which computes paths between points (or graph nodes) based on a cost model and can be used to ensure that corridor layouts are consistent with navigability requirements.[\[450\]](#)[\[451\]](#) More generally, proposed room placements may be evaluated by “scoring” them according to distances implied by a graph layout procedure, linking spatial placement decisions to the underlying navigation graph.[\[449\]](#)

Beyond basic routing, procedural methods frequently incorporate reachability and shortest-path constraints to support lock-and-key progression and puzzle solvability. One line of work explicitly treats dungeon verification as a graph problem: given graph topology and gameplay constraints, a heuristic can be used to help validate procedurally generated, graph-based dungeons.[\[452\]](#) A related guarantee used in dungeon generation is to construct a graph that ensures all rooms are connected (and therefore reachable), so that the level remains navigable as content is added.[\[479\]](#) For progression puzzles, lock-and-key generation frameworks use ordered sequences of keys and locked doors along walks, where the walk proceeds by taking a shortest available step that satisfies the current lock/key state.[\[480\]](#) In this way, the topology is not only connected but also constrained so that critical progression paths exist and can be completed under the game's gating rules.[\[480\]](#)

Topological characteristics can also be described at the level of standard graph families (e.g., tree, ring, or star structures) as inputs to the generator, with the expectation that certain structural patterns will better support reachability and puzzle design.[\[453\]](#) Additional work emphasizes determining whether placement of obsta-

cles is “safe” in the sense that it does not block required movement: after rooms are connected, the game uses a pathfinding algorithm to decide where obstacles can be placed without breaking intended navigation and accessibility.[\[472\]](#) More advanced evaluation approaches may consider multiple candidate routes, such as comparing dungeon paths against a “critical path” (the shortest required sequence to complete the level), to assess whether the generated topology and constraints preserve solvable navigation.[\[482\]](#)

Binary Space Partitioning (BSP) for level/space partitioning (why use it)

Binary Space Partitioning (BSP) is commonly used in game level and space partitioning because it supports fast, iterative construction of environments by recursively dividing space into convex regions. This makes BSP well suited to workflows where a designer or generator repeatedly refines layouts rather than producing a single static plan.[\[1\]](#) In dungeon-style procedural generation, the layout process often begins with a BSP-based spatial division; afterward, rooms are placed according to the partition structure and additional rules are applied to further determine geometry and gameplay-relevant objects.[\[2\]\[3\]](#)

Beyond simple subdivision, BSP can be integrated into broader procedural approaches that also analyze connectivity and navigability and then apply post-processing steps to keep the resulting architecture coherent. For example, one described pipeline uses BSP for space division, graph-based analysis to evaluate connections between regions, and post-processing to ensure that the final design remains consistent.[\[4\]](#) Related research has also proposed algorithms that rely on BSP partitioning by planes defined through polygonal faces, highlighting BSP’s role as a geometric method for organizing complex spatial or architectural data.[\[4\]](#)

VGLC dataset usage for 2D dungeon generation (formats & gaps)

The Video Game Level Corpus (VGLC) is a dataset of game levels represented in multiple formats, intended to support tasks such as analysis, imitation, and generation of game layouts.[\[131\]](#) In the context of 2D dungeon generation, VGLC can be used as a starting point to learn or derive structural patterns common to dungeon-like environments (e.g., room connectivity and spatial layouts), which can then be transferred to a procedural generator.[\[131\]\[132\]](#)

Formats and how they help dungeon generation

VGLC provides level data in more than one representation, including formats that can support different modeling approaches. For example, work describing VGLC notes that dungeon levels from well-known games can appear in Vector formats, which are useful when a generator or model needs explicit geometric structure rather than rasterized tiles.[\[129\]](#) Having multiple encodings also helps practitioners choose representations that best match their pipeline—such as using higher-level geometric

primitives for layout assembly or using alternative encodings for learning-based approaches.[\[131\]](#)

Using VGLC to construct “gaps” (unexplored layout space)

Because VGLC includes thousands of levels across different games, it can help designers identify where certain dungeon-generation patterns are common and where they are underrepresented. One approach is to treat the corpus as a set of examples and then deliberately sample or enforce constraints that push generation toward layout variations that are rare in the dataset, effectively creating “solid gaps” in the generated content relative to typical training distributions.[\[131\]](#) This is conceptually aligned with the broader role of procedural generation in creating unpredictable, varied levels across runs, as seen in dungeon-crawler and roguelike systems.[\[130\]](#)

Practical workflow for 2D dungeon generation pipelines

A typical workflow is to use VGLC data to (1) study room and corridor structure from existing dungeons, then (2) apply that structure within a procedural generator that assembles rooms using rules or spatial partitioning. Procedural dungeon generation is often framed as an initial step in level creation and can be implemented as a rule-based system that “pieces together” handcrafted room templates.[\[132\]](#)[\[133\]](#) VGLC-derived layout statistics can inform those rules—such as preferred room sizes, adjacency patterns, or corridor-styling constraints—so that generated 2D dungeons remain consistent with known dungeon geometry while still exploring new combinations.[\[131\]](#)

Related implementations

In addition to dataset-driven approaches, several GitHub-based dungeon-generation tools illustrate common technique patterns (e.g., rule-based generation, spatial partitioning, and automated assembly into playable layouts). These can complement VGLC usage by providing implementable generators into which corpus-derived constraints or learned heuristics can be inserted.[\[133\]](#)[\[134\]](#)[\[135\]](#) For example, Binary-Space Partitioning–based dungeon generators paired with editable room templates support configurable layouts, which makes them suitable targets for “gap” exploration driven by how VGLC layouts are clustered or characterized.[\[134\]](#)

Procedural puzzle room generation (algorithms / ML integration)

Puzzle room archetypes (room types, entrances/exits)

Puzzle rooms in game dungeons can be generated and classified using consistent archetypes, typically centered on (1) how the room’s internal traversal puzzle works and (2) how the room connects to adjacent spaces through entrances and exits. Procedural content generation (PCG) methods are often applied at this granularity—generating room-scale structures such as floorplans and room contents rather

than only whole levels—by first establishing a room “skeleton” and then adding puzzle-specific features or encounters iteratively.[\[383\]](#)[\[388\]](#)

One common archetype is the maze-like puzzle room, where the navigable space inside the room is itself the challenge. Maze generation approaches can be adapted to room-scale layouts by carving passages within solid regions and producing coherent, solvable structures at runtime; for example, a “maze generator” can be implemented as a randomized flood-fill–style process applied to regions between walls, yielding a connected maze-like subspace.[\[387\]](#)[\[492\]](#) In practice, maze generation algorithms such as randomized cell-growth schemes (selecting random cells and expanding the maze by carving to neighboring cells) or classical “perfect maze” methods (mazes with a single solution) can be used to ensure the room has a clear, learnable structure.[\[493\]](#)[\[496\]](#)

Another archetype is the block-pushing / interaction puzzle room, associated with dungeon designs where progress depends on manipulating objects and triggering local mechanisms. In dungeon puzzles of this type, the player may solve challenges “on the spot,” using actions such as moving crates and pressing switches that affect doors, barriers, or traversal options elsewhere in the dungeon.[\[498\]](#)[\[499\]](#) These rooms can be treated as repeatable templates for PCG: a generator can place interactable objects (tools, blocks, switches) in a layout that creates a constrained solution path or unlock sequence, aligning with PCG research that evaluates puzzle generation systems and their perceived difficulty.[\[390\]](#)[\[391\]](#)

A related archetype is the gatekeeping / parameter-gated room, in which the room’s entry-to-exit connectivity is blocked until requirements are satisfied in other rooms. Dungeon design principles emphasize that dungeons should include paths that remain inaccessible until certain parameters (often controlled by puzzle solutions elsewhere) are met elsewhere in the dungeon.[\[500\]](#) This archetype naturally fits room-based PCG: the floorplan generator can create a network of rooms and connections, then assign puzzles or requirements so that the dungeon’s critical paths become solvable only after the correct set of earlier room interactions have been completed.[\[388\]](#)[\[389\]](#)

Finally, puzzle rooms can be differentiated by their entrances and exits, especially in dungeon-to-world transitions. Dungeon “entrance” rooms are typically the junction that connects the dungeon interior to the overworld (world map), while interior puzzle rooms use entrances/exits to structure forward progression and enforce the intended dependency chain of puzzle solutions.[\[498\]](#) Across archetypes, PCG systems often work by producing room structures and then adding room-specific content—additional encounters, puzzle logic, and gated connectivity—to ensure that the overall dungeon traversal remains consistent with the intended puzzle dependency design.
[\[383\]](#)[\[388\]](#)

Topology + pathfinding integration (topo checks, connectivity, reachability)

Topology and pathfinding are commonly integrated into procedural dungeon generation by treating the dungeon as a graph whose nodes represent rooms (or

room areas) and whose edges represent potential connections (corridors, doors, or passages). After generating a coarse layout, algorithms typically validate that the resulting topology supports the intended traversal properties—such as global connectivity, reachability from a start region, and the ability to satisfy progression constraints (e.g., doors or objectives). This validation step is often described as a “topo check” over the generated map before final geometry is produced or before gameplay-critical features are instantiated.[\[428\]](#)

One widely used approach begins with creating a spatial partitioning of the level, such as Binary Space Partitioning (BSP), where a map is recursively split either horizontally or vertically to form a set of regions that can later be turned into rooms and hallways. Because BSP yields a hierarchical structure, it can be paired with graph construction and connectivity tests: after connections between subregions are proposed, the resulting room graph can be checked for connectivity and reachability from designated start/end rooms.[\[424\]](#)[\[427\]](#) This is particularly useful when the generator must ensure that all rooms needed for progression are reachable without breaking the intended layout logic.

Another approach models the dungeon’s traversability more directly using randomized flood-fill methods. For example, a maze generator can be implemented as a randomized flood fill on each solid region between rooms, producing corridors and passage structures while retaining a traversal-oriented structure. When combined with an explicit graph representation, the flood-filled connectivity naturally supports reachability checks, making it easier to ensure that the dungeon forms a coherent navigable space rather than disconnected pockets.[\[425\]](#)

Step-by-step procedural tutorials frequently include explicit start/end placement followed by subsequent connectivity logic. In such workflows, generators place start and end rooms arbitrarily (subject to minimum distances or constraints) and then generate intermediate connectivity to ensure the topology is traversable.[\[426\]](#) In practice, that often involves building a reachability graph (e.g., via BFS/DFS over corridors/edges) and rejecting or repairing generations that fail reachability constraints.

At a higher level of abstraction, some frameworks describe the overall pipeline as two complementary methods: one that maps the abstract graph into the required game space and another that generates geometry for the rooms and hallways “as required.”[\[428\]](#) In these pipelines, topology verification usually occurs after the graph mapping stage and before (or during) geometry instantiation, ensuring that the spatial realization still preserves the required connectivity and traversal properties.

Finally, topology constraints can be incorporated into the search or optimization process itself. For instance, evolutionary algorithms have been proposed to procedurally generate dungeon maps that include progression-related mission elements (such as locked-door missions), where the fitness evaluation can implicitly or explicitly reward topologies that remain solvable under the game’s progression rules.[\[429\]](#) In such methods, reachability is not merely validated after generation; it can guide the algorithm toward layouts whose connectivity supports the required lock-and-key or mission flow.

Research gap identification & evaluation strategy for Zeldastyle PCG

Research on “Zeldastyle” procedural content generation (PCG) shows a recurring mismatch between what is technically feasible and what has been empirically demonstrated. Although prior work discusses the appeal of using PCG to create a procedurally generated take on , existing implementations and studies appear limited, leaving notable gaps in both coverage of Zelda-specific design constraints and in documented player-facing outcomes.[\[156\]](#) This gap is reinforced by the observation that ’s appeal is strongly tied to its hand-crafted world and levels, where curated content is a key source of meaning and gameplay texture—making it unclear how well current PCG approaches replicate that value.[\[158\]](#)

A further research gap concerns the relationship between “generation coverage” and “Zelda-like experience.” One line of work suggests that generated content can enable more complete exploration than classic, fixed layouts, implying that PCG might improve traversal and discovery opportunities relative to hand-built level sets.[\[157\]](#) However, this claim does not fully address how Zelda-style structure (e.g., dungeon pacing, topology, and navigation rhythms) should be evaluated beyond exploration depth—especially when Zelda is treated as a reference for functional similarity rather than a direct blueprint.[\[159\]](#) As a result, evaluation strategies for Zeldastyle PCG remain under-specified: it is not enough to show that levels are generated; it must also be demonstrated that the generated levels sustain Zelda-like play patterns and production-quality content density.

To address these gaps, a plausible evaluation and identification strategy for Zeldastyle PCG can be built around (1) establishing a Zelda-inspired target level form, (2) selecting measurable properties aligned with Zelda’s gameplay expectations, and (3) comparing against hand-crafted baselines. For the target form, prior research has used a classic 2D top-down dungeon template inspired by roguelike conventions and functionally similar to , providing a concrete basis for reproducible generation and evaluation.[\[159\]](#) For measurable properties, evaluation can incorporate exploration-related outcomes—such as how thoroughly players can explore generated areas—because evidence suggests PCG may support more complete exploration than traditional layouts.[\[157\]](#) For baselines, hand-crafted Zelda content should be used to identify where generated levels underperform in “content value,” since hand-crafted levels are explicitly argued to add substantial content beyond what a single generated level may provide.[\[158\]](#) Finally, gaps can be identified through targeted comparisons between the generator’s outputs and known Zelda design patterns, including whether the procedural method captures the intended “Zelda-like” structure rather than producing levels that are only loosely analogous.[\[155\]\[156\]](#)

Encoding and validating “pacing signals” across seeds

In Zelda-like procedural content generation, “pacing signals” such as tension/relief cycles, sightline-driven discovery, and gated progression can be encoded on top of a dungeon’s basic connectivity graph. One approach begins by constructing a graph over the centers of rooms and connecting them using Delaunay triangulation, producing a connected structure that can later be constrained by additional gameplay logic.[\[71\]](#) Several related algorithms then map this graph into the game’s space while a separate procedure generates the geometry of rooms and hallways, enabling pacing-aware layout decisions to persist beyond raw connectivity.[\[72\]](#)

To encode gating and progression pacing, the level design process can explicitly represent lock–key relationships and tie them to specific encounter structures. For example, a lock can be resolved by providing an appropriate key item, such as a potion of poison immunity that prevents or mitigates an encounter with giant poisonous rats.[\[73\]](#) In this setting, dungeon state and encounter selection can also be influenced by whether keys are placed alongside “room key” and encounter logic, and by how locks or disabled traps affect subsequent random encounters.[\[74\]](#) More generally, systems that resolve locks and keys can act as decision points for event triggers so that random events (and their likelihoods) respond to the player’s progress and interactions with gates.[\[75\]](#)

At the metric level, pacing can be controlled by evaluating both macro-structure (e.g., “lock-key depth” and related loop-count parameters) and multiple micro-pacing components. One described scheme includes parameters for lock-key depth and loop count alongside three micro-pacing terms—such as safe weight, treasure weight, and distance influence—used to assess controllability of pacing during generation.[\[76\]](#) The same pipeline can mark key nodes representing locations the party will pass through before applying further generation steps, ensuring that pacing-critical locations are explicitly tracked rather than emerging only incidentally.[\[77\]](#) Gated progression can be implemented directly through structural constraints on the connectivity graph—for instance, creating partitions connected by a single corridor so that one partition’s rooms can be locked away until the player obtains the required key.[\[78\]](#)

Validation across many seeds focuses on whether these pacing relationships survive regeneration. Because pacing signals are represented through explicit graph-based constraints (locks/keys, partitioning corridors) and parameterized pacing metrics (depth, loop count, and micro-pacing weights), they can be checked by analyzing generated layouts for structural gate placement, correct reachability of locked areas, and the intended progression rhythm under repeated runs.[\[76\]\[78\]](#) Similarly, by tying encounter generation and event triggering probabilities to the same lock–key system, generators can be tested to ensure that pacing-relevant difficulty and event patterns remain consistent in aggregate, rather than drifting when only geometry or connectivity is regenerated.[\[74\]\[75\]](#)

Paper submission and peer review

In academic work on procedural content generation (PCG) for game levels, submissions to peer-reviewed venues typically undergo formal editorial and refereeing procedures prior to acceptance. For example, venues that hosted studies on generating

“Zelda”-style action-adventure levels reported that submitted manuscripts underwent thorough peer review and that accepted papers were specifically included among the set of “Zelda-style action adventures” results.[\[216\]](#)[\[221\]](#) Some reports also describe the refereeing model in procedural terms, such as the use of double-blind peer review for submissions.[\[218\]](#)[\[219\]](#) In addition, while some accompanying materials may not have been subject to peer review (e.g., certain figures or supplementary content), the central submitted manuscripts were still reviewed through the journal or conference peer-review process.[\[217\]](#)

References

- [1]: [Procedural Generation: An Overview | by Kenny - Medium](#)
- [2]: [Procedural generation - Wikipedia](#)
- [3]: [A Very Short History of Dynamic and Procedural Content Generation](#)
- [4]: [Procedural Content Generation for video games, a friendly approach](#)
- [5]: [Procedural Content Generation in Games: A Survey with Insights on ...](#)
- [6]: [\[PDF\] Procedural Content Generation: Techniques and Applications](#)
- [7]: [Is AI Used for Procedural Content Gen in Gaming? - Lenovo](#)
- [8]: [Utilizing Gen AI for Procedural Content Generation in video games](#)
- [9]: [Procedural Content Generation in Games | Springer Nature Link](#)
- [10]: [Procedural Content Generation Overview | Unreal Engine ...](#)
- [11]: [Procedural content generation in games](#)
- [12]: [Procedural Content Generation: An Overview](#)
- [13]: [Procedural Content Generation for video games, a friendly approach](#)
- [14]: [Procedural Content Generation in Games: A Survey with Insights on ...](#)
- [15]: [Is AI Used for Procedural Content Gen in Gaming? - Lenovo](#)
- [16]: [Runtime Evaluation of Procedural Content Generation in an Endless ...](#)
- [17]: [On the Evaluation of Procedural Level Generation Systems](#)
- [18]: [\[PDF\] PROCEDURAL DUNGEON GENERATION - DiVA portal](#)
- [19]: [Agentic PCG: Procedural Content Generation via Tool-using LLMs](#)
- [20]: [Procedural Content Generation via Machine Learning](#)
- [21]: [MOPCGRL: Multi-Objective Procedural Content Generation via ...](#)
- [22]: [Anyone know a guaranteed-solvable 'small key' dungeon ...](#)
- [23]: [Is there a practical solution to generating Zelda-style key ...](#)
- [24]: [Lock and Key Dungeons - BorisTheBrave.Com](#)
- [25]: [Keep Running | Procedural Level Generation in Sure Footing](#)
- [26]: [The Procedural Content Generation Benchmark](#)
- [27]: [Runtime Evaluation of Procedural Content Generation in an Endless ...](#)
- [28]: [AI agent evaluation: frameworks, metrics & testing strategies - Algolia](#)
- [29]: [\[PDF\] PROCEDURAL DUNGEON GENERATION - DiVA portal](#)

- [30]: [Agentic PCG: Procedural Content Generation via Tool-using LLMs](#)
- [31]: [Procedural Content Generation via Machine Learning](#)
- [32]: [MOPCGRL: Multi-Objective Procedural Content Generation via ...](#)
- [33]: [Anyone know a guaranteed-solvable 'small key' dungeon ...](#)
- [34]: [The Secret Behind Unexplored: Cyclic Dungeon Generation ...](#)
- [35]: [Is there a practical solution to generating Zelda-style key ...](#)
- [36]: [Lock and Key Dungeons - BorisTheBrave.Com](#)
- [37]: [Flow in Procedural Generation](#)
- [38]: [A Hybrid Approach to Procedural Generation of Roguelike ...](#)
- [39]: [Unreal Engine Construction Script Guidelines](#)
- [40]: [How to Approach Run-time generation in Unreal Engine](#)
- [41]: [Procedural Generation of Semantically Correct Levels in ...](#)
- [42]: [Keep Running | Procedural Level Generation in Sure Footing](#)
- [43]: [The Procedural Content Generation Benchmark](#)
- [44]: [Runtime Evaluation of Procedural Content Generation in an Endless ...](#)
- [45]: [\[PDF\] PROCEDURAL DUNGEON GENERATION - DiVA portal](#)
- [46]: [Agentic PCG: Procedural Content Generation via Tool-using LLMs](#)
- [47]: [Anyone know a guaranteed-solvable 'small key' dungeon ...](#)
- [48]: [Is there a practical solution to generating Zelda-style key ...](#)
- [49]: [Lock and Key Dungeons - BorisTheBrave.Com](#)
- [50]: [Flow in Procedural Generation](#)
- [51]: [Procedural Generation of Semantically Correct Levels in ...](#)
- [52]: [Procedural generation of dungeons' maps and locked-door ...](#)
- [53]: [Procedural Dungeon Generation Algorithm Explained](#)
- [54]: [Dungeon Generation in Unexplored - BorisTheBrave.Com](#)
- [55]: [Time Pressure, or, How to Run Any Dungeon : r/dndnext](#)
- [56]: [\(PDF\) Procedural Generation of Dungeons](#)
- [57]: [Everything's a Dungeon: A Different Approach to ...](#)
- [58]: [Constructive Generation Methods for Dungeons and Levels](#)
- [59]: [Procedural Content Generation framework \(UE 5.2+\) : r/unrealengine](#)
- [60]: [Fix: Unreal PCG Determinism Different Results Across Runs - Bugnet](#)
- [61]: [Procedural Content Generation Overview | Unreal Engine 5.8 ...](#)
- [62]: [Procedural Generation in Games: Algorithms, Techniques & Examples](#)
- [63]: [Dungeon Generation in Unexplored - BorisTheBrave.Com](#)
- [64]: [\(PDF\) Procedural Generation of Dungeons - ResearchGate](#)
- [65]: [\[PDF\] UC Santa Cruz - eScholarship.org](#)
- [66]: [Procedural Dungeon Generation Algorithm Explained](#)
- [67]: [Constructive Generation Methods for Dungeons and Levels](#)

- [68]: [Dungeon Generation in Unexplored - BorisTheBrave.Com](#)
- [69]: [Everything's a Dungeon: A Different Approach to ...](#)
- [70]: [Constructive Generation Methods for Dungeons and Levels](#)
- [71]: [Procedural Dungeon Generation Algorithm Explained](#)
- [72]: [Constructive Generation Methods for Dungeons and Levels](#)
- [73]: [Dungeon Generation in Unexplored - BorisTheBrave.Com](#)
- [74]: [Time Pressure, or, How to Run Any Dungeon : r/dndnext](#)
- [75]: [Exploration RULES!](#)
- [76]: [\(PDF\) Procedural Generation of Dungeons](#)
- [77]: [Everything's a Dungeon: A Different Approach to ...](#)
- [78]: [Constructive Generation Methods for Dungeons and Levels](#)
- [79]: [What The Legend of Zelda could learn from Rogue Heroes](#)
- [80]: [Procedural generation of dungeons' maps and locked-door missions ...](#)
- [81]: [Procedural Content Generation in Games: A Survey with Insights on ...](#)
- [82]: [Procedural terrain in the Style of "Zelda: A Link to the Past"](#)
- [83]: [\[BoTW\] Thoughts on a procedurally-generated game in the style of ...](#)
- [84]: [Effects of The Legend of Zelda: Breath of the Wild and Studio Ghibli ...](#)
- [85]: [Procedural Dungeon Generation: A Survey - Journals](#)
- [86]: [An introduction to procedural lock and key dungeon generation](#)
- [87]: [\[PDF\] A Hybrid Approach to Procedural Generation of Roguelike Video ...](#)
- [88]: [What is the difference between level design and narrative design in ...](#)
- [89]: [\[PDF\] Field Level Design in "The Legend of Zelda: Breath of the Wild"](#)
- [90]: [\[PDF\] Procedural Zelda: A PCG Environment for Player Experience ...](#)
- [91]: [An introduction to procedural lock and key dungeon generation](#)
- [92]: [An introduction to procedural lock and key dungeon generation](#)
- [93]: [Anyone know a guaranteed-solvable 'small key' dungeon algorithm?](#)
- [94]: [Lock and Key Dungeons - BorisTheBrave.Com](#)
- [95]: [Procedural Dungeon Generation Node Graph Lock And Key - belafly](#)
- [96]: [\(PDF\) Procedural Dungeon Generation: A Survey - ResearchGate](#)
- [97]: [Unexplored's Secret: 'Cyclic Dungeon Generation' - Game Developer](#)
- [98]: [The Legend of Zelda: Skyward Sword's dungeon design](#)
- [99]: [The Minish Cap's dungeon design | Boss Keys](#)
- [100]: [Lock and Key Dungeons - BorisTheBrave.Com](#)
- [101]: [In terms of pathfinding, are the more interesting dungeons ...](#)
- [102]: [The Legend of Zelda / Zelda II's dungeon design | Boss Keys](#)
- [103]: [The Legend of Zelda: The Wind Waker's dungeon design.](#)
- [104]: [DURING, like Zelda, or AFTER, like Megaman? | ZD Forums](#)
- [105]: [Procedural generation of dungeons' maps and locked-door ...](#)

- [106]: [Towards Objective Metrics for Procedurally Generated ...](#)
- [107]: [Procedural Generation in Games - Computerphile](#)
- [108]: [An introduction to procedural lock and key dungeon ...](#)
- [109]: [An Exploration of Procedural Methods in Game level design](#)
- [110]: [When does procedural generation work well in games, and ...](#)
- [111]: [Procedural Generation of Semantically Correct Levels in ...](#)
- [112]: [\(PDF\) Towards Objective Metrics for Procedurally ...](#)
- [113]: [Dungeon Generation in Unexplored - BorisTheBrave.Com](#)
- [114]: [Level Design in Procedural Generation](#)
- [115]: [Procedural Level Generation in Sure Footing \(AI and Games ...](#)
- [116]: [Level layouts procedurally generated from arbitrary input ...](#)
- [117]: [Constructive Generation Methods for Dungeons and Levels](#)
- [118]: [A Hybrid Approach to Procedural Generation of Roguelike ...](#)
- [119]: [Learning From The Masters: Level Design In The Legend ...](#)
- [120]: [Understanding Flow in Level Design | by Mannan Soni](#)
- [121]: [What Is a Critical Path in Project Management?](#)
- [122]: [Critical Path Percentage - Help Center - SmartPM](#)
- [123]: [Level Design Principles](#)
- [124]: [Critical Path Method For Large-Scale Projects](#)
- [125]: [Critical Path Method: the complete guide for 2026](#)
- [126]: [Procedural game progression dependency graphs - runevision](#)
- [127]: [Procedural Generation: An Overview | by Kenny - Medium](#)
- [128]: [Procedural generation: a primer for game devs - Game Developer](#)
- [129]: [The VGLC: The Video Game Level Corpus](#)
- [130]: [Towards Game Level Generation Through LLM and GAN](#)
- [131]: [DOOM Level Generation using Generative Adversarial ...](#)
- [132]: [Procedural Dungeon Generator | Future Data Lab](#)
- [133]: [procedural-generation · GitHub Topics](#)
- [134]: [dungeon · GitHub Topics](#)
- [135]: [A Procedurally Generated GitHub CLI Roguelike Where ...](#)
- [136]: [Procedural dungeon code for Unity3D](#)
- [137]: [Level Design in Procedural Generation - Game Developer](#)
- [138]: [Zelda games' environmental storytelling through level design](#)
- [139]: [ELI5: How is procedural generation different from just regular ...](#)
- [140]: [\[PDF\] Field Level Design in "The Legend of Zelda: Breath of the Wild"](#)
- [141]: [Data-Driven Level Generation for Action Role Playing Games](#)
- [142]: [Procedural generation of dungeons' maps and locked-door missions ...](#)
- [143]: [\[PDF\] A Hybrid Approach to Procedural Generation of Roguelike Video ...](#)

- [144]: [Procedural Generation - Zelda Universe Forums](#)
- [145]: [Dungeon Generation: A Procedural Generation Guide](#)
- [146]: [Dungeon Generation in Unexplored - BorisTheBrave.Com](#)
- [147]: [An introduction to procedural lock and key dungeon generation](#)
- [148]: [Cyclic Dungeon Generation by Sersa Victory - itch.io](#)
- [149]: [Need some input on how to improve procedural dungeon generation](#)
- [150]: [EPC2016 | Cyclic Dungeon Generation | Joris Dormans - YouTube](#)
- [151]: [Procedural Dungeon Generator | Future Data Lab](#)
- [152]: [What Makes a Great Zelda Dungeon? : r/truezelda](#)
- [153]: [An open world with linear dungeons? | ZD Forums](#)
- [154]: [How the Dungeons of The Legend of Zelda Challenge Players ...](#)
- [155]: [An introduction to procedural lock and key dungeon ...](#)
- [156]: [The Zelda-Rogue Design Trap](#)
- [157]: [Procedural generation of dungeons' maps and locked-door ...](#)
- [158]: [Procedurally Assisted Generation](#)
- [159]: [A Hybrid Approach to Procedural Generation of Roguelike ...](#)
- [160]: [A Hybrid Approach to Procedural Generation of Roguelike ...](#)
- [161]: [Procedural Dungeon Generation: A Survey](#)
- [162]: [Quest to Dungeon \(QtD\): Towards a Tool that Supports ...](#)
- [163]: [\[ALL\] Which game had the best pace? : r/zelda - Reddit](#)
- [164]: [An introduction to procedural lock and key dungeon generation](#)
- [165]: [An introduction to procedural lock and key dungeon generation](#)
- [166]: [Constructive Generation Methods for Dungeons and Levels](#)
- [167]: [Procedural generation of dungeons' maps and locked-door missions ...](#)
- [168]: [Dungeon Generation in Unexplored - BorisTheBrave.Com](#)
- [169]: [\(PDF\) Procedural Generation of Dungeons - ResearchGate](#)
- [170]: [The Zelda-Rogue Design Trap](#)
- [171]: [Dungeon Generation in Unexplored - BorisTheBrave.Com](#)
- [172]: [Need some input on how to improve procedural dungeon ...](#)
- [173]: [\(PDF\) Procedural Generation of Dungeons](#)
- [174]: [Unexplored's Secret: 'Cyclic Dungeon Generation'](#)
- [175]: [RL Weekly](#)
- [176]: [Insights into ModelAgnostic MetaLearning on ...](#)
- [177]: [The Obstacle Tower: A Generalization Challenge in Vision, ...](#)
- [178]: [FLAVOR TEXT GENERATION FOR ROLE-PLAYING ...](#)
- [179]: [Kannada Continuous Speech Recognition Using Deep ...](#)
- [180]: [\[25WS - BA\] Using Pacing to Control Procedural Content Generation ...](#)
- [181]: [High Dimensional Procedural Content Generation - arXiv](#)

- [182]: [Pacing & beats \[~LEVEL DESIGN #17\] - YouTube](#)
- [183]: [\[PDF\] Procedural Content Generation: An Overview - Game AI Pro](#)
- [184]: [Level Progression Design and Pacing : r/gamedesign - Reddit](#)
- [185]: [\[PDF\] Level Generation with Constrained Expressive Range](#)
- [186]: [\[PDF\] Procedural Level Design for Platform Games](#)
- [187]: [\[PDF\] Procedural Content Generation - Cornell University](#)
- [188]: [Room Generation using Constraint Satisfaction - pvigier's blog](#)
- [189]: [Spatial Layout of Procedural Dungeons Using Linear Constraints ...](#)
- [190]: [\[PDF\] Fast Procedural Level Population with Playability Constraints](#)
- [191]: [\[PDF\] Spatial Layout of Procedural Dungeons Using Linear Constraints ...](#)
- [192]: [Sturgeon-MKIII: Simultaneous Level and Example Playthrough ...](#)
- [193]: [\[PDF\] A constraint-based method for solving sequential manipulation ... - MIT](#)
- [194]: [Level Design in Procedural Generation](#)
- [195]: [An introduction to procedural lock and key dungeon ...](#)
- [196]: [Traditional level design VS procedural level design?](#)
- [197]: [An introduction to procedural lock and key dungeon generation](#)
- [198]: [Dungeon Generation in Unexplored - BorisTheBrave.Com](#)
- [199]: [An Exploration of Procedural Methods in Game level design](#)
- [200]: [A mixed-initiative design framework for procedural content ...](#)
- [201]: [The Zelda-Rogue Design Trap](#)
- [202]: [Procedural Content Generation: An Overview](#)
- [203]: [Exploring Procedural Content Generation for a 2D Space ...](#)
- [204]: [On the Evaluation of Procedural Level Generation Systems](#)
- [205]: [\(PDF\) A modular framework for automated evaluation of ...](#)
- [206]: [BiLSTM and dynamic fuzzy AHP-GA method for procedural ...](#)
- [207]: [\[Literature Review\] The Procedural Content Generation ...](#)
- [208]: [PlainQAFact: Automatic Factuality Evaluation Metric for Biomedical ...](#)
- [209]: [Automated Metrics vs Human Evaluation in AI | Label Studio](#)
- [210]: [Procedural Content Generation in Games: A Survey with Insights on ...](#)
- [211]: [\(PDF\) Procedural Content Generation in Games \(Guest Editorial\)](#)
- [212]: [Integrating Procedural and Handmade Level Design - Academia.edu](#)
- [213]: [Creating a Newer and Improved Procedural Content Generation ...](#)
- [214]: [\[PDF\] Procedural Content Generation for Emergent Gameplay](#)
- [215]: [Procedural generation - Wikipedia](#)
- [216]: [Guest Editorial: Procedural Content Generation in Games](#)
- [217]: [Procedural Generation of Semantically Correct Levels in ...](#)
- [218]: [Submissions | Game Nexus](#)
- [219]: [Experimental Game Workshop call for proposals at GDC ...](#)

[220]: [Start small: Training controllable game level generators ...](#)

[221]: [Guest Editorial: Procedural Content Generation in Games](#)

[222]: [\[PDF\] CG 2018 - MAASTRICHT Proceedings of the 2018 IEEE Conference ...](#)

[223]: [\[PDF\] EXTENDED ABSTRACTS | Brown CS](#)

[224]: [Reviews by manonamora - The Interactive Fiction Database](#)

[225]: [Adolescent Masculinity and the Geek Aesthetic: A Study of Gaming ...](#)

[226]: [12th International Conference on Interactive Digital Storytelling ...](#)

[227]: [\[PDF\] A Hybrid Approach to Procedural Generation of Roguelike Video ...](#)

[228]: [Introduction to Procedural Content Generation | PDF | Expert - Scribd](#)

[229]: [\[PDF\] Field Level Design in "The Legend of Zelda: Breath of the Wild"](#)

[230]: [Stylized Zelda Landscape! Large Scale PCG Tutorial with Gaea ...](#)

[231]: [Paper Database - EXAG Workshop](#)

[232]: [Guided Game Level Repair via Explainable AI - arXiv](#)

[233]: [Analysis of Robustness of a Large Game Corpus - ACM Digital Library](#)

[234]: [\[PDF\] Leveraging Procedural Generation to Benchmark Reinforcement ...](#)

[235]: [PCG node placement on a 3D navigation mesh. Node count = 107.](#)

[236]: [\[PDF\] Level generation for cooperative games](#)

[237]: [Playable level from generated game progression dependency ...](#)

[238]: [G-PCGRL: Procedural Graph Data Generation via ...](#)

[239]: [Procedural game progression dependency graphs - runevision](#)

[240]: [Graph-based Generation of Action-Adventure Dungeon ...](#)

[241]: [An introduction to procedural lock and key dungeon generation](#)

[242]: [An introduction to procedural lock and key dungeon generation](#)

[243]: [Lock and Key Dungeons - BorisTheBrave.Com](#)

[244]: [Playable level from generated game progression dependency graph ...](#)

[245]: [A Hybrid Approach to Procedural Generation of Roguelike Video ...](#)

[246]: [\[PDF\] Shortest Walk in a Dungeon Graph](#)

[247]: [Room Generation using Constraint Satisfaction](#)

[248]: [Spatial Layout of Procedural Dungeons Using Linear ...](#)

[249]: [The Dungeon Variations Problem Using Constraint ... - DROPS](#)

[250]: [\(PDF\) The Dungeon Variations Problem Using Constraint ...](#)

[251]: [Procedural Constraint-based Generation for Game ...](#)

[252]: [Graph Rewriting for Procedural Level Generation](#)

[253]: [Procedural Dungeon Generation Algorithm Explained](#)

[254]: [What is the most appropriate path-finding solution for a ...](#)

[255]: [An introduction to graph rewriting for procedural content ...](#)

[256]: [Shortest Walk in a Dungeon Graph](#)

[257]: [Procedural Path Generation In A Grid](#)

- [258]: [Understanding region segmented hierarchical pathfinding](#)
- [259]: [Creating natural paths on terrains using pathfinding - runevision](#)
- [260]: [HPA* \(hierarchical pathfinding algorithm\) is impressive in ...](#)
- [261]: [Multi-threaded Recast-Based A* Pathfinding for Scalable ...](#)
- [262]: [Pathfinding Tutorial - Understanding the Basics of Pathfinding](#)
- [263]: [Pathfinding with Hard Constraints](#)
- [264]: [Procedural Generation in GMS #6: A* Is Born \(Pathfinding's Greatest ...](#)
- [265]: [Moving Through a Procedural Dungeon; error when I hit the edges.](#)
- [266]: [Procedural Generation Placing Spawner - Page 2 - Game Engine ...](#)
- [267]: [Creating natural paths on terrains using pathfinding - Blog - runevision](#)
- [268]: [Procedurally Generated Dungeons - VAZGRIZ](#)
- [269]: [How to fix pathfinding for AI in a procedural generated dungeon?](#)
- [270]: [Pathfinding best practices and surprising uses](#)
- [271]: [How to know if there is no path between the source and destination ...](#)
- [272]: [Pathfinding via precomputed tables - TADS](#)
- [273]: [Cannot find path from this location : r/Borderlands - Reddit](#)
- [274]: [Dungeon Generation in Unexplored - BorisTheBrave.Com](#)
- [275]: [An introduction to procedural lock and key dungeon generation](#)
- [276]: [PROCEDURAL DUNGEON GENERATION](#)
- [277]: [Pathfinding with procedural generation - Help - Godot Forum](#)
- [278]: [Procedural Generation in GMS #6: A* Is Born \(Pathfinding's Greatest ...](#)
- [279]: [Procedural map generation: preventing blocked maps? - Reddit](#)
- [280]: [Simplified 2D Grid PATHFINDING in Godot 4.2 - YouTube](#)
- [281]: [Procedural terrain in the Style of "Zelda: A Link to the Past"](#)
- [282]: [Tactical Combat 23 - Tile Neighbors For A* Pathfinding](#)
- [283]: [Tutorial for procedural level generation : r/gamemaker](#)
- [284]: [Procedural Level Generation - Steven Annunziato](#)
- [285]: [Level Design in Procedural Generation](#)
- [286]: [Procedural Generation: An Overview | by Kenny - Medium](#)
- [287]: [Procedural Content Generation: An Overview](#)
- [288]: [Procedural Content Generation Overview | Unreal Engine ...](#)
- [289]: [How do Randomly Generated Levels Work? : r/gamedesign](#)
- [290]: [Dungeon Generation in Unexplored - BorisTheBrave.Com](#)
- [291]: [An introduction to procedural lock and key dungeon ...](#)
- [292]: [Level Design in Procedural Generation](#)
- [293]: [Dungeon Generation: A Procedural Generation Guide](#)
- [294]: [Procedural generation of dungeons' maps and locked-door ...](#)
- [295]: [Runtime Evaluation of Procedural Content Generation in ...](#)

- [296]: [\[22WS - BA\] Creating Engaging Game Spaces ... - BayernCollab](#)
- [297]: [Procedural Content Generation via Machine Learning](#)
- [298]: [Procedural Content Generation in Games: A Survey with Insights on ...](#)
- [299]: [\[AI Challenge\] Procedural Content Generated Challenge Discussion...](#)
- [300]: [Procedural Content Generation via Machine Learning](#)
- [301]: [Multi-task procedural content generation with reinforcement learning](#)
- [302]: [Procedural Generation: An Overview | by Kenny - Medium](#)
- [303]: [Research: Procedural Content Generation - Antonios Liapis](#)
- [304]: [\[PDF\] Procedural Content Generation: An Overview - Game AI Pro](#)
- [305]: [Level Design in Procedural Generation - Game Developer](#)
- [306]: [Procedural Generation: An Overview | by Kenny - Medium](#)
- [307]: [Procedural generation - Wikipedia](#)
- [308]: [How can procedural generation be used to create meaningful quest ...](#)
- [309]: [\[PDF\] PROCEDURAL GENERATION OF GAME-WORLDS](#)
- [310]: [Procedural Content Generation in Games: A Survey with Insights on ...](#)
- [311]: [Procedural worlds: The simulation, functional and planning ...](#)
- [312]: [\[PDF\] An Exploration of Procedural Methods in Game level design](#)
- [313]: [Procedural Content Generation in Games: A Survey ...](#)
- [314]: [View of Procedural Content Generation in Games](#)
- [315]: [Hybrid Approaches to Procedural Content Generation for ...](#)
- [316]: [Procedural content generation for games: A survey](#)
- [317]: [Procedural Content Generation for Game Designers](#)
- [318]: [Dungeon Generation in Unexplored - BorisTheBrave.Com](#)
- [319]: [An introduction to procedural lock and key dungeon generation](#)
- [320]: [The Zelda-Rogue Design Trap - Medium](#)
- [321]: [Cyclic Dungeon Generation by Sersa Victory - itch.io](#)
- [322]: [\[PDF\] Mixed-Initiative Procedural Generation of Dungeons Using Game ...](#)
- [323]: [Procedural generation of dungeons' maps and locked-door missions ...](#)
- [324]: [What The Legend of Zelda could learn from Rogue Heroes](#)
- [325]: [Procedural Zelda-Like Adventures on Mobile | Gamers - Vocal Media](#)
- [326]: [What are some ideal algorithms for Rogue-like 2D dungeon ...](#)
- [327]: [Graph-based dungeon generator: improvement ideas?](#)
- [328]: [An introduction to graph rewriting for procedural content ...](#)
- [329]: [Graph Rewriting for Procedural Level Generation](#)
- [330]: [Procedural Dungeon Generation Connect Rooms With ...](#)
- [331]: [Designing Procedurally Generated Levels](#)
- [332]: [Best approach for connecting doors for procedural ...](#)
- [333]: [Graph-based Generation of Action-Adventure Dungeon ...](#)

[334]: [Procedural Generation: Graph Rewriting](#)

[335]: [Graph-based dungeon generator: improvement ideas? - Reddit](#)

[336]: [\(PDF\) Procedural generation of dungeon maps, missions and rooms](#)

[337]: [Help with procedural level generation \(different room sizes\)](#)

[338]: [algorithm - Procedural... house with rooms generator](#)

[339]: [Procedural generation of dungeons' maps and locked-door ...](#)

[340]: [Constrained Level Generation through Grammar-Based ...](#)

[341]: [2-Step Evolutionary Algorithm for the generation of ...](#)

[342]: [Procedural Generation of Semantically Correct Levels in ...](#)

[343]: [Automatic Video Game Level Generation](#)

[344]: [\(PDF\) A procedural procedural level generator generator](#)

[345]: [Procedural Content Generation in Games: A Survey ...](#)

[346]: [Procedural generation of dungeons' maps and locked-door missions ...](#)

[347]: [Procedural generation of dungeons' maps and locked-door missions ...](#)

[348]: [\[PDF\] Evolving lock-and-key puzzles based on nonlinear player ...](#)

[349]: [Procedural dungeon generation for a 2D top-down game](#)

[350]: [An introduction to procedural lock and key dungeon ...](#)

[351]: [Lock and Key Dungeons - BorisTheBrave.Com](#)

[352]: [Shortest Walk in a Dungeon Graph](#)

[353]: [Spatial Layout of Procedural Dungeons Using Linear ...](#)

[354]: [Procedural Dungeon Generation Node Graph Lock And Key](#)

[355]: [Graph-based generation of action-adventure dungeon ...](#)

[356]: [Spatial Layout of Procedural Dungeons Using Linear ...](#)

[357]: [Graph Dungeon Generator](#)

[358]: [Topological Machine Learning II - YouTube](#)

[359]: [\[PDF\] Enhancing Graph Representation Learning with Localized ...](#)

[360]: [Topological Methods in Machine Learning: A Tutorial for Practitioners](#)

[361]: [\[PDF\] Boosting Graph Pooling with Persistent Homology - NIPS](#)

[362]: [a variational framework for graph generation with fine-grained...](#)

[363]: [A Topology Layer for Machine Learning | SAIL Blog - Stanford AI Lab](#)

[364]: [Procedural Content Generation via Generative Artificial ...](#)

[365]: [Deep learning for procedural content generation](#)

[366]: [Procedural Content Generation in Games: A Survey with ...](#)

[367]: [Procedural Content Generation for video games, a friendly ...](#)

[368]: [List of Accepted Papers](#)

[369]: [Daily Papers - Hugging Face](#)

[370]: [CGAL - The Computational Geometry Algorithms Library](#)

[371]: [confviz/frontend/src/data/papers/icml.json at master - GitHub](#)

[372]: [\[PDF\] Development of trustworthy intelligent avatars in virtual immersion](#)

[373]: [\[PDF\] Course Detailed Syllabus \(B.Tech.\). School of Computer Science ...](#)

[374]: [\[PDF\] The Journal of Computing Sciences in Colleges](#)

[375]: [An introduction to procedural lock and key dungeon generation](#)

[376]: [Dungeon Generation in Unexplored - BorisTheBrave.Com](#)

[377]: [\[PDF\] A Novel Procedural Generation for Level Design of Mansions and ...](#)

[378]: [Procedurally Generating Locked Doors](#)

[379]: [Procedural Dungeon Generation Algorithm Explained : r/gamedev](#)

[380]: [\[PDF\] PROCEDURAL DUNGEON GENERATION - DiVA portal](#)

[381]: [Procedural Dungeon Generation Algorithm Explained](#)

[382]: [An introduction to graph rewriting for procedural content ...](#)

[383]: [Algorithm for environmental puzzles? \(Doors and switches etc.\)](#)

[384]: [Procedural Levels Generation — Puzzle Game](#)

[385]: [The Case for Procedural Generation in Puzzle Games - YouTube](#)

[386]: [\[PDF\] Procedural Puzzle Generation: A Survey](#)

[387]: [Rooms and Mazes: A Procedural Dungeon Generator - Bob Nystrom](#)

[388]: [Dungeon Generation in Unexplored - BorisTheBrave.Com](#)

[389]: [Constructive Generation Methods for Dungeons and Levels](#)

[390]: [Mixed-initiative Puzzle Design Tool for Everyone Must Die](#)

[391]: [Procedural Level Generation with Difficulty ...](#)

[392]: [Book Review: Procedural Generation in Game Design](#)

[393]: [Procedural Content Generation in Games: A Survey ...](#)

[394]: [The Zelda-Rogue Design Trap](#)

[395]: [A Hybrid Approach to Procedural Generation of Roguelike ...](#)

[396]: [Procedural Content Generation in Games : r/gamedev](#)

[397]: [Procedural Content Generation in Games \(Computational ...](#)

[398]: [Level Design in Procedural Generation](#)

[399]: [Procedural Content Generation for Games](#)

[400]: [Procedural terrain in the Style of "Zelda: A Link to the Past"](#)

[401]: [An introduction to procedural lock and key dungeon ...](#)

[402]: [An introduction to procedural lock and key dungeon generation](#)

[403]: [Dungeon Generation in Unexplored - BorisTheBrave.Com](#)

[404]: [Level Design in Procedural Generation](#)

[405]: [Constructive Generation Methods for Dungeons and Levels](#)

[406]: [Is there a practical solution to generating Zelda-style key ...](#)

[407]: [Lock and Key Dungeons - BorisTheBrave.Com](#)

[408]: [An introduction to procedural lock and key dungeon ...](#)

[409]: [Graph Dungeon Generator](#)

- [410]: [Shortest Walk in a Dungeon Graph](#)
- [411]: [Graph-based Generation of Action-Adventure Dungeon ...](#)
- [412]: [An introduction to procedural lock and key dungeon ...](#)
- [413]: [Lock and Key Dungeons - BorisTheBrave.Com](#)
- [414]: [Procedural Dungeon Generation Algorithm Explained](#)
- [415]: [Graph-based Generation of Action-Adventure Dungeon ...](#)
- [416]: [Spatial Layout of Procedural Dungeons Using Linear ...](#)
- [417]: [Constructive Generation Methods for Dungeons and Levels](#)
- [418]: [Dungeon Generation in Unexplored - BorisTheBrave.Com](#)
- [419]: [Procedural Generation Puzzle Design : r/roguelikedev - Reddit](#)
- [420]: [Designing Procedural Dungeon Layouts with PCG Functions in ...](#)
- [421]: [Procedural generation of dungeons' maps and locked-door missions ...](#)
- [422]: [Beginning Game Development: Procedural Dungeon Generation](#)
- [423]: [Rooms and Mazes: A Procedural Dungeon Generator - Bob Nystrom](#)
- [424]: [Looking for a simple dungeon generation algorithm](#)
- [425]: [Rooms and Mazes: A Procedural Dungeon Generator](#)
- [426]: [Procedurally generating dungeons using predefined rooms](#)
- [427]: [Procedural Dungeon Generation with Binary Space Partitioning!](#)
- [428]: [Constructive Generation Methods for Dungeons and Levels](#)
- [429]: [Procedural generation of dungeons' maps and locked-door ...](#)
- [430]: [What is faster? ProceduralMeshComponent or BSP?](#)
- [431]: [Constructive Generation Methods for Dungeons and Levels](#)
- [432]: [A Novel Procedural Generation for Level Design of ...](#)
- [433]: [isaacfvi/BSP-Procedural-generation](#)
- [434]: [BSP-fields: An exact representation of polygonal objects by ...](#)
- [435]: [Procedural Dungeon Generation with Binary Space Partitioning!](#)
- [436]: [Dungeon Generation using Binary Space Trees](#)
- [437]: [Basic BSP Dungeon generation](#)
- [438]: [Constructive Generation Methods for Dungeons and Levels](#)
- [439]: [A Technical Note on BSP-Based Dungeon Design](#)
- [440]: [A Novel Procedural Generation for Level Design of ...](#)
- [441]: [Constructive Generation Methods for Dungeons and Levels](#)
- [442]: [Day 1: Dungeon Generation and BSP trees](#)
- [443]: [\[PDF\] PROCEDURAL DUNGEON GENERATION - DiVA portal](#)
- [444]: [Constructive Generation Methods for Dungeons and Levels](#)
- [445]: [\[PDF\] Combining Constructive Procedural Dungeon Generation Methods ...](#)
- [446]: [\[PDF\] Algorithms used for procedurally generated dungeons - DiVA portal](#)
- [447]: [Dungeon Generation using Binary Space Trees](#)

[448]: [Dungeon generation using BSP trees | eskerda.com](#)

[449]: [Graph-based dungeon generator: improvement ideas? - Reddit](#)

[450]: [\[PDF\] PROCEDURAL DUNGEON GENERATION - DiVA portal](#)

[451]: [Procedural Dungeon Generation Connect Rooms With Passageways](#)

[452]: [\[PDF\] Shortest Walk in a Dungeon Graph](#)

[453]: [\(PDF\) Procedural Generation of Dungeons - ResearchGate](#)

[454]: [Constructive Generation Methods for Dungeons and Levels](#)

[455]: [Playable level from generated game progression dependency ...](#)

[456]: [Procedural Generation: An Overview | by Kenny - Medium](#)

[457]: [High Dimensional Procedural Content Generation - arXiv](#)

[458]: [An introduction to graph rewriting for procedural content generation](#)

[459]: [An introduction to graph rewriting for procedural content generation](#)

[460]: [\[PDF\] EVALUATING RELIABILITY IN LLM-BASED PROCEDURAL ...](#)

[461]: [\[PDF\] Procedural Content Generation - Cornell University](#)

[462]: [Symmetry-Aware LLM-Driven Generation and Repair of Interactive ...](#)

[463]: [\[PDF\] 5.1 Validation of Graph-Based Models \(Analysis and Consistency\)](#)

[464]: [Stuck in the Middle: Generating Levels without \(or with\) ...](#)

[465]: [Designing A Procedurally Generated Metroidvania Style ...](#)

[466]: [Deep Learning for PCG of 2D video game levels](#)

[467]: [Automatic generation and evaluation of platform games](#)

[468]: [Procedural Content Generation via Tool-using LLMs](#)

[469]: [Simulation and Testing of A* Algorithm with Look-Ahead Capabil](#)

[470]: [\(PDF\) Debugging Grammars for Level Generation](#)

[471]: [\[PDF\] 5.1 Validation of Graph-Based Models \(Analysis and Consistency\)](#)

[472]: [Level Design in Procedural Generation - Game Developer](#)

[473]: [An introduction to procedural lock and key dungeon generation](#)

[474]: [Procedural Levels Generation — Puzzle Game](#)

[475]: [Traditional level design VS procedural level design? : r/IndieGaming](#)

[476]: [\[PDF\] Interactions between game design and procedural level generation](#)

[477]: [Constructive Generation Methods for Dungeons and Levels](#)

[478]: [An introduction to procedural lock and key dungeon generation](#)

[479]: [Procedural Dungeon Generation Algorithm Explained](#)

[480]: [Shortest Walk in a Dungeon Graph](#)

[481]: [PROCEDURAL DUNGEON GENERATION](#)

[482]: [A Hybrid Approach to Procedural Generation of Roguelike ...](#)

[483]: [What elements make The Legend of Zelda's dungeon design stand out](#)

[484]: [What's a Zelda dungeon theme or concept that hasn't been ... - Reddit](#)

[485]: [What are some puzzle ideas for a big dungeon? - Facebook](#)

- [486]: [Easy ZELDA-Style Dungeon Design for D&D - YouTube](#)
- [487]: [Breath of the Wild - More Labyrinth-Like Dungeons | ZD Forums](#)
- [488]: [Building a Legend of Zelda style dungeon.](#)
- [489]: [What elements make The Legend of Zelda's dungeon ...](#)
- [490]: [The Ultimate Methodology of creating Zelda Dungeon Level](#)
- [491]: [Rooms and Mazes: A Procedural Dungeon Generator](#)
- [492]: [Rooms and Mazes: A Procedural Dungeon Generator](#)
- [493]: [Maze Generation Algorithms - An Exploration](#)
- [494]: [Procedural Generation and Random Maze Generation](#)
- [495]: [Maze generation algorithm](#)
- [496]: [What's a good algorithm to generate a maze? \[closed\]](#)
- [497]: [What Legend of Zelda Can Teach Us about Dungeon Design](#)
- [498]: [Depicting the Level Design of a Legend of Zelda Dungeon](#)
- [499]: [Dungeons with Zelda-esque Design? : r/rpg](#)
- [500]: [Zelda Temples and "Realistic" Dungeon Design - The Cauldron](#)
- [501]: [How to generate perfect mazes?](#)
- [502]: [Maze-solving algorithm](#)
- [503]: [How to Generate Perfect Mazes?](#)
- [504]: [Maze generations: Algorithms and Visualizations.](#)
- [505]: [Maze Theory](#)
- [506]: [This is not a maze : r/proceduralgeneration](#)
- [507]: [Randomized Depth-First-Search Algorithm for Maze ...](#)
- [508]: [Maze Generation Algorithms: A Comparative Research ...](#)
- [509]: [Maze generation algorithms \(Prim, Kruskal, flood fill\)](#)
- [510]: [Maze Theory](#)
- [511]: [Maze generation algorithm - Wikipedia](#)
- [512]: [Generating Mazes with Inductive Graphs | jelv.is](#)
- [513]: [Kim, Paul H - OhioLINK ETD](#)
- [514]: [\[PDF\] Maze Generation Algorithms: A Comparative Research with a Focus ...](#)
- [515]: [Maze Algorithms \(visual\) : r/compsci - Reddit](#)
- [516]: [A maze generation algorithm where I can choose entrance ...](#)
- [517]: [Is there a maze generation algorithm that includes loops ...](#)
- [518]: [Maze generation algorithms for existing grids](#)
- [519]: [Fundamentals of Maze Generation](#)